

Advanced Machine Learning

Lecture 13: LLM, Finetuning, Fast Inference

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

28 January 2025



Outline

Repetition

- Attention
- Transformer
- Diffusion models

Large Language Models (LLM)

- Models
- Finetuning
- Fast inference
- Downsides

A Warning

Outline

Repetition

- Attention
- Transformer
- Diffusion models

Large Language Models (LLM)

- Models
- Finetuning
- Fast inference
- Downsides

A Warning

Different types of attention

Attention:

- ▶ first in **natural language processing (NLP)**,
- ▶ then in **computer vision (CV)**.

Different types of attention:

- ▶ **global** and **local attention** (NLP, Luong, Pham, and Manning [22])
- ▶ **soft** and **hard attention** (CV, Xu et al. [35])
- ▶ Bahdanau, Cho, and Bengio [1] uses **retrieval system terminology**:

$$\text{attention}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \sum_j \text{similarity}(\mathbf{q}, \mathbf{k}_j) \cdot \mathbf{v}_j,$$

q “query”, **k** “key”, **v** “value”

- ▶ **additive** or **multiplicative** similarity

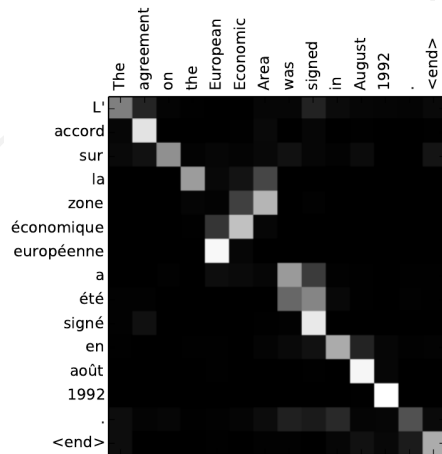
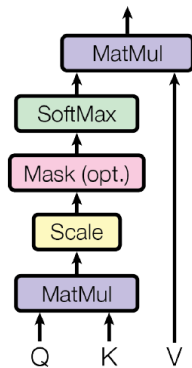


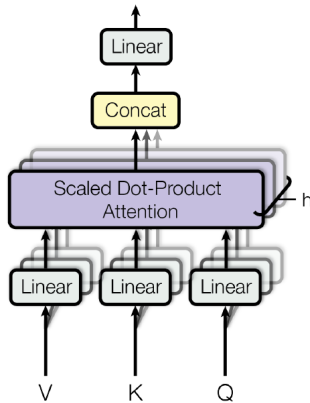
Figure taken from Bahdanau, Cho, and Bengio [1].

“Attention is all you need” (Vaswani et al. [33], 2017)

Scaled Dot-Product Attention



Multi-Head Attention



All figures taken from Vaswani et al. [33].

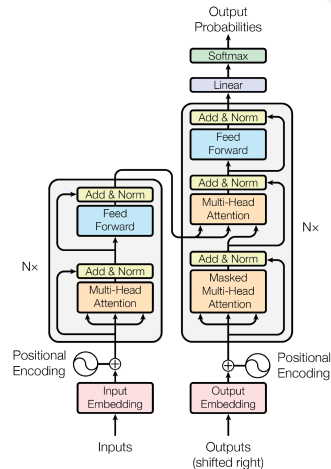


Figure 1: The Transformer - model architecture.

Encoder vs. decoder

Stack encoders: This idea was pursued by researchers at [Google](#) and results in [Bidirectional Encoder Representations from Transformers \(BERT\)](#), see Devlin et al. [8], new models are [RoBERTa](#), [DistilBERT](#), [ALBERT](#), [Bard](#), ...

Stack decoders: This idea was pursued by researchers at [OpenAI](#) and results in [Generative Pre-trained Transformer \(GPT\)](#):

[GPT](#): 2018, Radford et al. [27]

[GPT-2](#): 2019, Radford et al. [28]

[GPT-3](#): 2020, Brown et al. [3]

[GPT-3.5](#): 2022

[GPT-4](#): 2023, OpenAI [25]

Training of [ChatGPT](#): [pre-training](#) (self-supervised learning), [Q & A](#) (supervised learning), [reinforcement learning from human feedback](#) (RLHF), see Ouyang et al. [26]

Vision transformer

Vision transformer (ViT, Dosovitskiy et al. [9]) for **computer vision**.

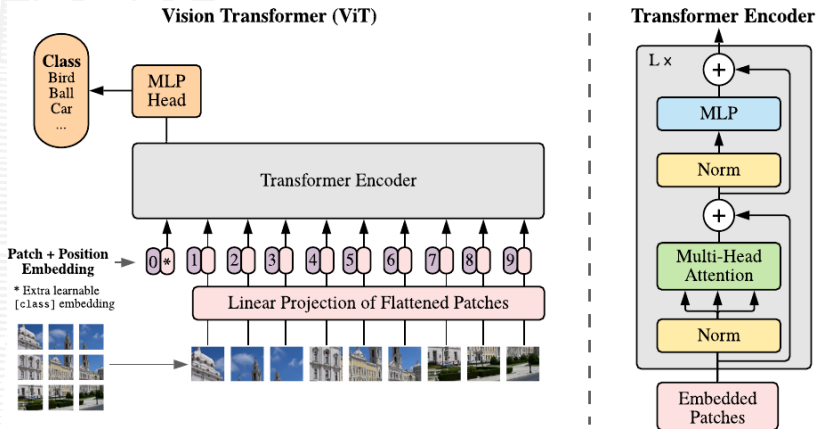


Image taken from Dosovitskiy et al. [9]

Latent diffusion model

Introduction to **diffusion models**: Higham, Higham, and Grindrod [15]. **Stable Diffusion** is based on the **latent diffusion model** (LDM, Rombach et al. [30]) by the CompVis group at LMU Munich.

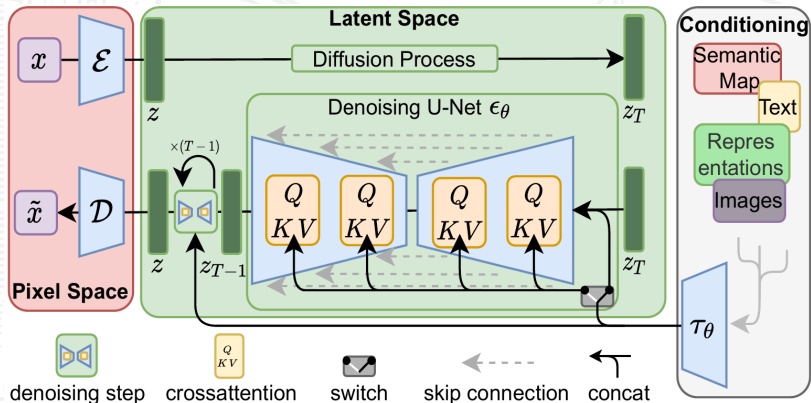


Image taken from Rombach et al. [30]

Outline

Repetition

- Attention
- Transformer
- Diffusion models

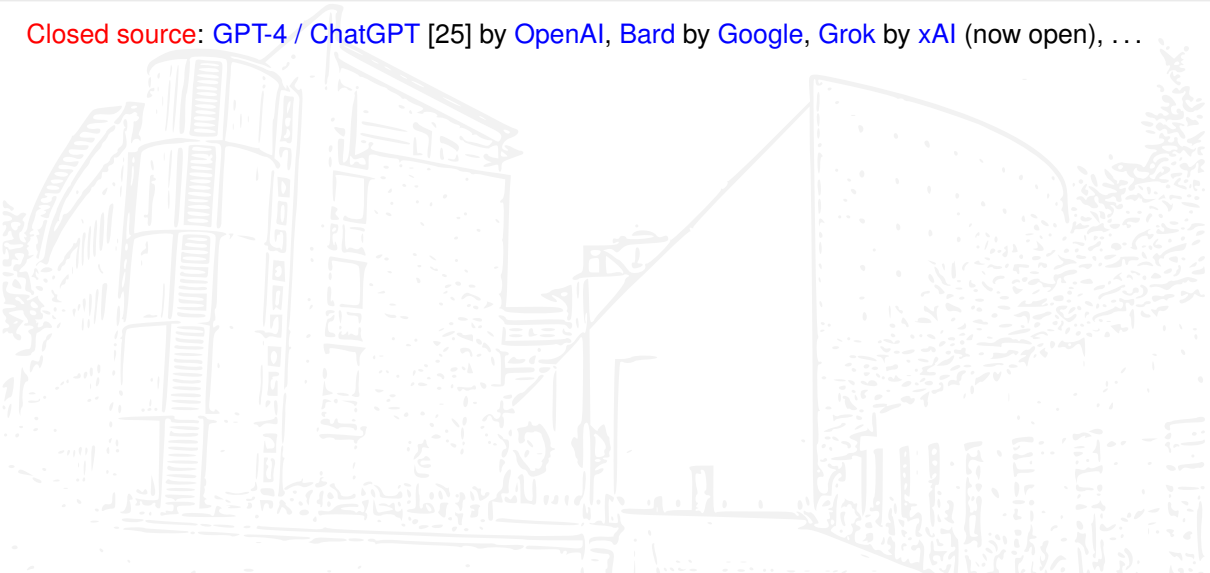
Large Language Models (LLM)

- Models
- Finetuning
- Fast inference
- Downsides

A Warning

Closed vs. open LLM

Closed source: GPT-4 / ChatGPT [25] by OpenAI, Bard by Google, Grok by xAI (now open), ...



Closed vs. open LLM

Closed source: GPT-4 / ChatGPT [25] by OpenAI, Bard by Google, Grok by xAI (now open), ...

First **open LLMs** by EleutherAI:

- ▶ GPT-Neo (March 2021), <https://github.com/EleutherAI/gpt-neo>,
- ▶ GPT-J (June 2021), <https://huggingface.co/EleutherAI/gpt-j-6b>

both trained on The Pile, a 825 GiB dataset collected by EleutherAI (Gao et al. [14]).

Closed vs. open LLM

Closed source: GPT-4 / ChatGPT [25] by OpenAI, Bard by Google, Grok by xAI (now open), ...

First **open LLMs** by EleutherAI:

- ▶ GPT-Neo (March 2021), <https://github.com/EleutherAI/gpt-neo>,
- ▶ GPT-J (June 2021), <https://huggingface.co/EleutherAI/gpt-j-6b>

both trained on The Pile, a 825 GiB dataset collected by EleutherAI (Gao et al. [14]).

Well known **families of “open” LLMs** have been published since 2023 by Meta:

- ▶ LLaMA 1 (February 2023, Touvron et al. [32]) available as LLaMA-7B, 13B, 30B, and 65B
- ▶ LLaMA 2 (July 2023, Touvron et al. [31]), available as 7B, 13B, and 70B (34B withheld)
- ▶ Llama 3 (April 2024), available as 8B and 70.6B
- ▶ Llama 3.1 (July 2024), Llama 3.2 (September 2024), Llama 3.3 (December 7, 2024)

Llama has a **special license** and the **training data is not provided**.

Closed vs. open LLM

Closed source: GPT-4 / ChatGPT [25] by OpenAI, Bard by Google, Grok by xAI (now open), ...

First **open LLMs** by EleutherAI:

- ▶ GPT-Neo (March 2021), <https://github.com/EleutherAI/gpt-neo>,
- ▶ GPT-J (June 2021), <https://huggingface.co/EleutherAI/gpt-j-6b>

both trained on The Pile, a 825 GiB dataset collected by EleutherAI (Gao et al. [14]).

Well known **families of “open” LLMs** have been published since 2023 by Meta:

- ▶ LLaMA 1 (February 2023, Touvron et al. [32]) available as LLaMA-7B, 13B, 30B, and 65B
- ▶ LLaMA 2 (July 2023, Touvron et al. [31]), available as 7B, 13B, and 70B (34B withheld)
- ▶ Llama 3 (April 2024), available as 8B and 70.6B
- ▶ Llama 3.1 (July 2024), Llama 3.2 (September 2024), Llama 3.3 (December 7, 2024)

Llama has a **special license** and the **training data is not provided**.

Further open LLMs exist, we mention Mistral 7B (September 2023), Mixtral 8x7B (December 2023), a **Mixture of Experts Model (MOE)** by Mistral AI. Details about the training are missing.

Model sizes & estimated training costs

The models become **larger** (Nova Premier by Amazon has **470B**, appears in 2024), smaller models become **better** (Microsoft: Phi-4 w/ & Phi-3 w/ 14B, Phi-2 w/ 2.7B, Phi-1.5 & Phi-1 w/ 1.3B).

Model sizes & estimated training costs

The models become **larger** (Nova Premier by Amazon has **470B**, appears in 2024), smaller models become **better** (Microsoft: Phi-4 w/ & Phi-3 w/ 14B, Phi-2 w/ 2.7B, Phi-1.5 & Phi-1 w/ 1.3B).

Microsoft calls the Phi models **small language models (SLM)**, others group LLMs into

Nano (e.g., Phi), **XS** (7B), **S** (20B), **M** (70B), **L** (180B).

Model sizes & estimated training costs

The models become **larger** (Nova Premier by Amazon has **470B**, appears in 2024), smaller models become **better** (Microsoft: Phi-4 w/ & Phi-3 w/ 14B, Phi-2 w/ 2.7B, Phi-1.5 & Phi-1 w/ 1.3B).

Microsoft calls the Phi models **small language models (SLM)**, others group LLMs into

Nano (e.g., Phi), **XS** (7B), **S** (20B), **M** (70B), **L** (180B).

Training LLMs is **expensive**, e.g., numbers for **pre-training** of LLaMA 2 (see **model card**):

	Time (GPU hours)	Power Consumption (W)	Carbon Emitted (tCO2eq)
LLaMA 2 7B	184.320	400	31.22
LLaMA 2 13B	368.640	400	62.44
LLaMA 2 70B	1.720.320	400	291.42
Total	3.311.616		539.00

Model sizes & estimated training costs

The models become **larger** (Nova Premier by Amazon has **470B**, appears in 2024), smaller models become **better** (Microsoft: Phi-4 w/ & Phi-3 w/ 14B, Phi-2 w/ 2.7B, Phi-1.5 & Phi-1 w/ 1.3B).

Microsoft calls the Phi models **small language models (SLM)**, others group LLMs into

Nano (e.g., Phi), **XS** (7B), **S** (20B), **M** (70B), **L** (180B).

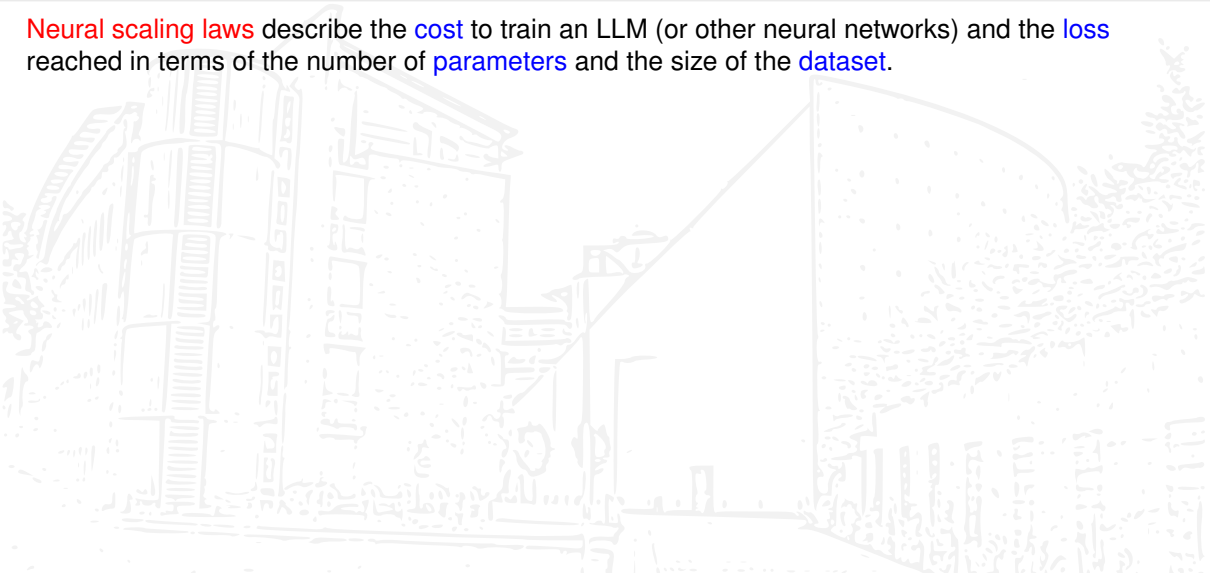
Training LLMs is **expensive**, e.g., numbers for **pre-training** of LLaMA 2 (see **model card**):

	Time (GPU hours)	Power Consumption (W)	Carbon Emitted (tCO2eq)
LLaMA 2 7B	184.320	400	31.22
LLaMA 2 13B	368.640	400	62.44
LLaMA 2 70B	1.720.320	400	291.42
Total	3.311.616		539.00

Training these models is impossible for small companies or individuals $\rightsquigarrow$ **Foundation Models**.

Neural scaling laws

Neural scaling laws describe the **cost** to train an LLM (or other neural networks) and the **loss** reached in terms of the number of **parameters** and the size of the **dataset**.



Neural scaling laws

Neural scaling laws describe the **cost** to train an LLM (or other neural networks) and the **loss** reached in terms of the number of **parameters** and the size of the **dataset**.

Kaplan et al. [18], 2020 **empirically** derived scaling laws in the form of **power laws** for LLM, later confirmed in the training of GPT-3.

Neural scaling laws

Neural scaling laws describe the **cost** to train an LLM (or other neural networks) and the **loss** reached in terms of the number of **parameters** and the size of the **dataset**.

Kaplan et al. [18], 2020 **empirically** derived scaling laws in the form of **power laws** for LLM, later confirmed in the training of GPT-3.

Chinchilla Scaling Law (2022, Hoffmann et al. [16]): Let N be the **number of parameters**, D the **size of the dataset**. Then the **loss** is given by

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

where E , A , and B are **constants** and **empirically**

$$\alpha = 0.34, \quad \beta = 0.28.$$

Neural scaling laws

Neural scaling laws describe the **cost** to train an LLM (or other neural networks) and the **loss** reached in terms of the number of **parameters** and the size of the **dataset**.

Kaplan et al. [18], 2020 **empirically** derived scaling laws in the form of **power laws** for LLM, later confirmed in the training of GPT-3.

Chinchilla Scaling Law (2022, Hoffmann et al. [16]): Let N be the **number of parameters**, D the **size of the dataset**. Then the **loss** is given by

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

where E , A , and B are **constants** and **empirically**

$$\alpha = 0.34, \quad \beta = 0.28.$$

In Liu and Tegmark [21] the authors argue based on “**Lottery Tickets**” (i.e., smaller subnetworks that train well, see Frankle and Carbin [10]) that $\alpha = 1/3$ might be the true value.

Remarks on alignment; classification and limits of LLM

Recap: LLM training divided into

- ▶ self-supervised learning / pre-training
- ▶ instruction finetuning
- ▶ alignment using reinforcement learning from human feedback (RLHF)

Remarks on alignment; classification and limits of LLM

Recap: LLM training divided into

- ▶ self-supervised learning / pre-training
- ▶ instruction finetuning
- ▶ alignment using reinforcement learning from human feedback (RLHF)

ChatGPT and LLaMA are aligned to minimize the amount of racism, sexism, violence, and profanity. Copyright restrictions are also trained into the models. Training the model to align often makes it worse in other tasks.

Remarks on alignment; classification and limits of LLM

Recap: LLM training divided into

- ▶ self-supervised learning / pre-training
- ▶ instruction finetuning
- ▶ alignment using reinforcement learning from human feedback (RLHF)

ChatGPT and LLaMA are aligned to minimize the amount of racism, sexism, violence, and profanity. Copyright restrictions are also trained into the models. Training the model to align often makes it worse in other tasks.

Some models, i.e., Grok and Mistral 7B are not aligned. Mistral AI publishes unaligned models and leaves a possible alignment to the users.

Remarks on alignment; classification and limits of LLM

Recap: LLM training divided into

- ▶ self-supervised learning / pre-training
- ▶ instruction finetuning
- ▶ alignment using reinforcement learning from human feedback (RLHF)

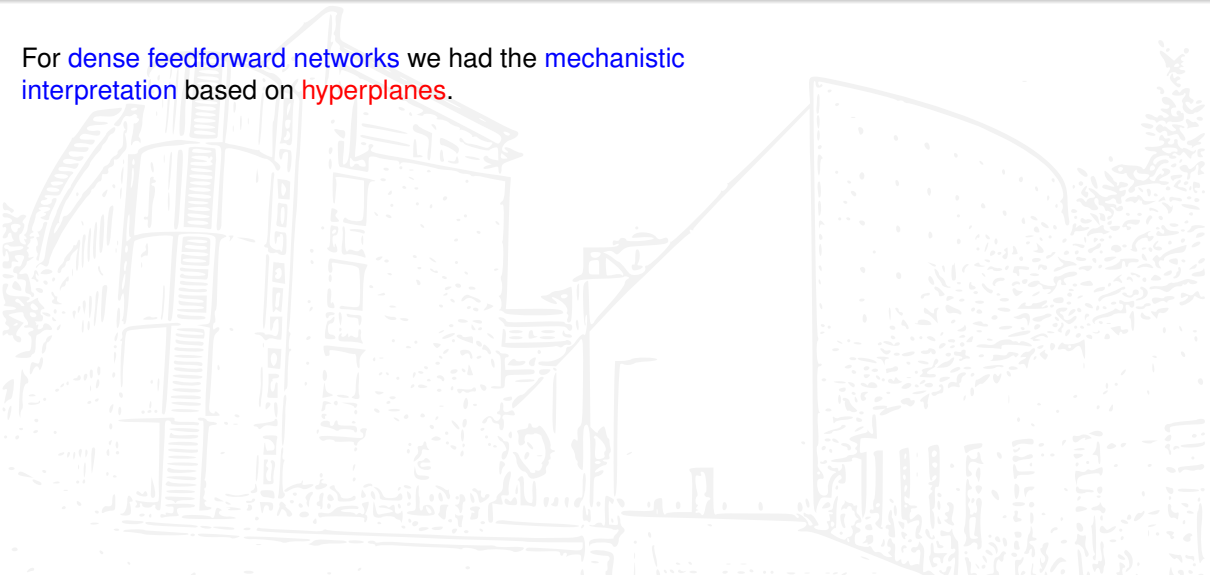
ChatGPT and LLaMA are aligned to minimize the amount of racism, sexism, violence, and profanity. Copyright restrictions are also trained into the models. Training the model to align often makes it worse in other tasks.

Some models, i.e., Grok and Mistral 7B are not aligned. Mistral AI publishes unaligned models and leaves a possible alignment to the users.

To clearly depict the details of different models, Mitchell et al. [23] invented so-called model cards in 2018. A criticism of the possibilities that LLMs offer is given in the “stochastic parrot” paper by Bender et al. [2] (Authors Timnit Gebru and Margaret Mitchell were fired by Google afterwards).

Interpretability: RASP, Tracr, & Transformer Programs

For dense feedforward networks we had the mechanistic interpretation based on hyperplanes.



Interpretability: RASP, Tracr, & Transformer Programs

For **dense feedforward networks** we had the **mechanistic interpretation** based on **hyperplanes**.

RNN are interpreted as **finite state machines** (e.g., Oliva and Lago-Fernández [24]).

Interpretability: RASP, Tracr, & Transformer Programs

For **dense feedforward networks** we had the **mechanistic interpretation** based on **hyperplanes**.

RNN are interpreted as **finite state machines** (e.g., Oliva and Lago-Fernández [24]).

For **transformer networks** there is something similar:

- ▶ **Restricted Access Sequence Processing Language (RASP)** as simple **programming language** (2021, Weiss, Goldberg, and Yahav [34])

Interpretability: RASP, Tracr, & Transformer Programs

For **dense feedforward networks** we had the **mechanistic interpretation** based on **hyperplanes**.

RNN are interpreted as **finite state machines** (e.g., Oliva and Lago-Fernández [24]).

For **transformer networks** there is something similar:

- ▶ **Restricted Access Sequence Processing Language (RASP)** as simple **programming language** (2021, Weiss, Goldberg, and Yahav [34])
- ▶ **Tracr** as **compiler** for RASP (2023, Lindner et al. [20])

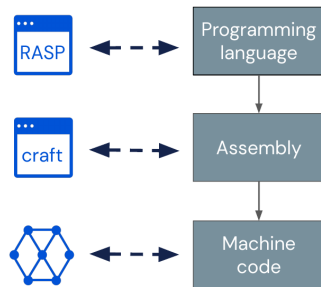


Figure 3: Tracr translates RASP to Craft and then to model weights, analogous to how programming languages are first translated to assembly then to machine code.

Figure and caption taken from Lindner et al. [20].

Interpretability: RASP, Tracr, & Transformer Programs

For **dense feedforward networks** we had the **mechanistic interpretation** based on **hyperplanes**.

RNN are interpreted as **finite state machines** (e.g., Oliva and Lago-Fernández [24]).

For **transformer networks** there is something similar:

- ▶ **Restricted Access Sequence Processing Language (RASP)** as simple **programming language** (2021, Weiss, Goldberg, and Yahav [34])
- ▶ **Tracr** as **compiler** for RASP (2023, Lindner et al. [20])
- ▶ **Transformer Programs**: modified Transformer, after training converted into a discrete, **human-readable program** (2023, Friedman, Wettig, and Chen [13])

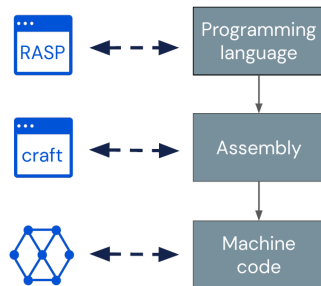


Figure 3: Tracr translates RASP to Craft and then to model weights, analogous to how programming languages are first translated to assembly then to machine code.

Figure and caption taken from Lindner et al. [20].

LLM as operating system

Prompting as **look-up of data** in **latent space**? Andrej Karpathy on Twitter / X:

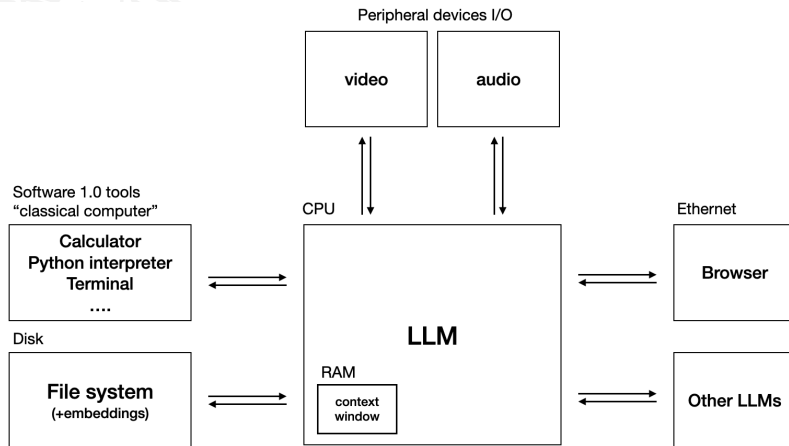
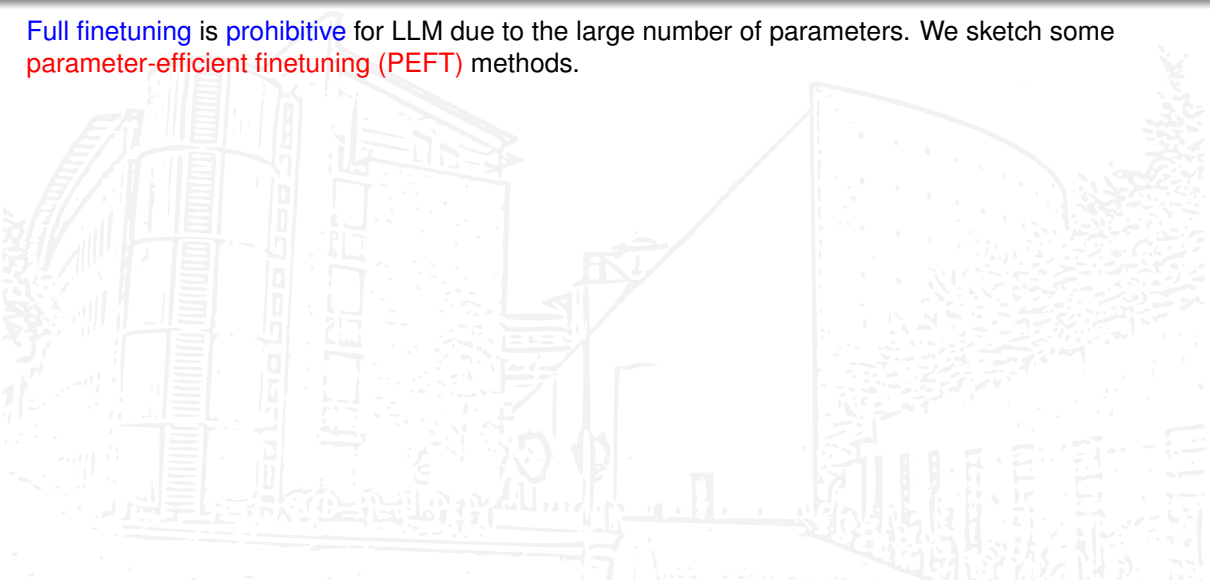


Image taken from <https://twitter.com/karpathy/status/1723140519554105733>.

Parameter-efficient finetuning (PEFT)

Full finetuning is prohibitive for LLM due to the large number of parameters. We sketch some parameter-efficient finetuning (PEFT) methods.



Parameter-efficient finetuning (PEFT)

Full finetuning is prohibitive for LLM due to the large number of parameters. We sketch some parameter-efficient finetuning (PEFT) methods.

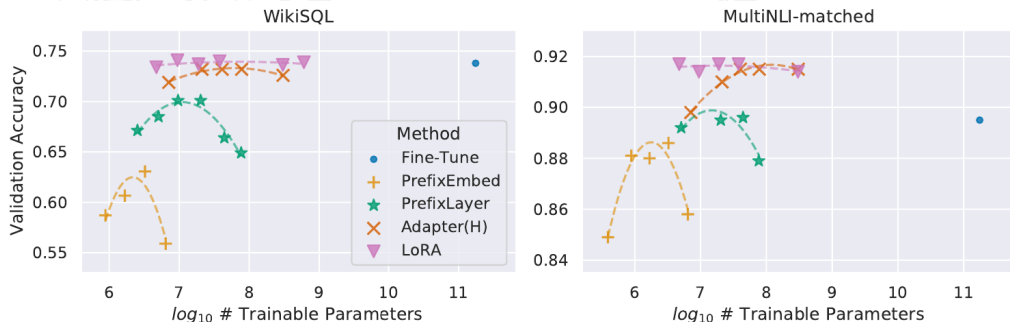
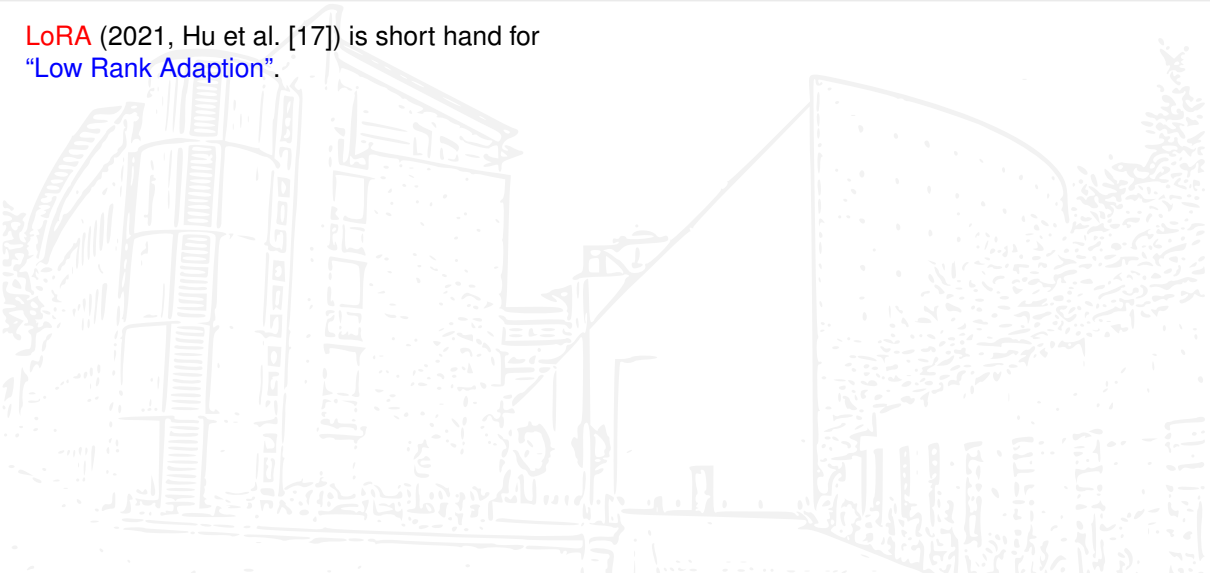


Figure 2: GPT-3 175B validation accuracy vs. number of trainable parameters of several adaptation methods on WikiSQL and MNLI-matched. LoRA exhibits better scalability and task performance. See [Section F.2](#) for more details on the plotted data points.

Figure and description taken from Hu et al. [17].

PEFT: LoRA

LoRA (2021, Hu et al. [17]) is short hand for “Low Rank Adaption”.



PEFT: LoRA

LoRA (2021, Hu et al. [17]) is short hand for “Low Rank Adaption”.

Full finetuning of $\mathbf{W} \in \mathbb{R}^{d \times d}$ uses weight updates $\Delta \mathbf{W} \in \mathbb{R}^{d \times d}$ with d^2 elements,

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \Delta \mathbf{W}.$$

PEFT: LoRA

LoRA (2021, Hu et al. [17]) is short hand for “Low Rank Adaption”.

Full finetuning of $\mathbf{W} \in \mathbb{R}^{d \times d}$ uses weight updates $\Delta \mathbf{W} \in \mathbb{R}^{d \times d}$ with d^2 elements,

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \Delta \mathbf{W}.$$

LoRA is based on low-rank updates

$$\Delta \mathbf{W} = \mathbf{B}\mathbf{A}, \quad \mathbf{A} \in \mathbb{R}^{r \times d}, \quad \mathbf{B} \in \mathbb{R}^{d \times r}$$

with $2rd$ parameters for $r \ll d$.

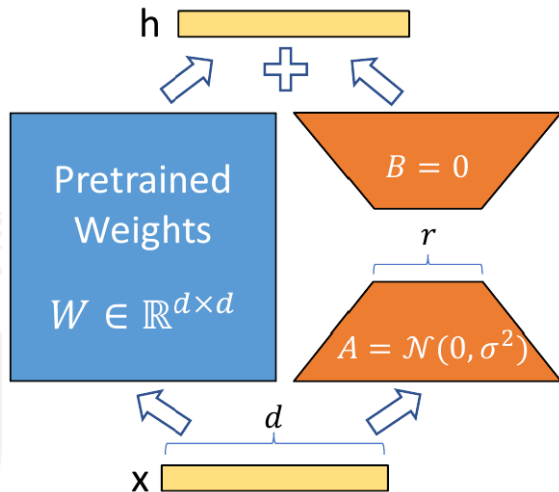


Figure taken from Hu et al. [17].

PEFT: LoRA

LoRA (2021, Hu et al. [17]) is short hand for “Low Rank Adaption”.

Full finetuning of $\mathbf{W} \in \mathbb{R}^{d \times d}$ uses weight updates $\Delta \mathbf{W} \in \mathbb{R}^{d \times d}$ with d^2 elements,

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \Delta \mathbf{W}.$$

LoRA is based on low-rank updates

$$\Delta \mathbf{W} = \mathbf{B}\mathbf{A}, \quad \mathbf{A} \in \mathbb{R}^{r \times d}, \quad \mathbf{B} \in \mathbb{R}^{d \times r}$$

with $2rd$ parameters for $r \ll d$.

Initially, **B** is zero. Ordinary backpropagation is used to determine the updates. After training, updates can be fused.

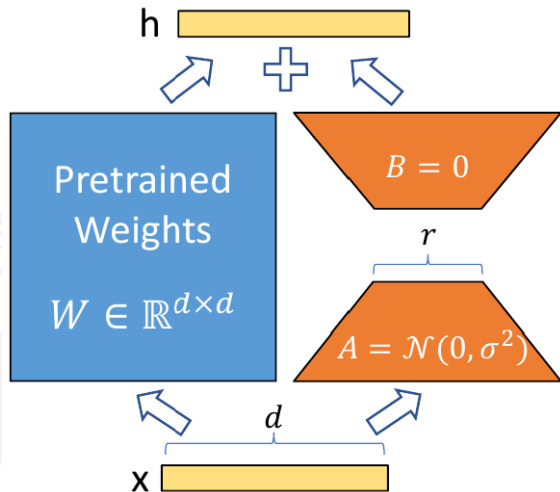


Figure taken from Hu et al. [17].

PEFT: VeRA

The number of trainable parameters can be reduced further. An example is **VeRA**, vector-based random matrix adaption (2023, Kopiczko, Blankevoort, and Asano [19]), $n + r$ parameters:

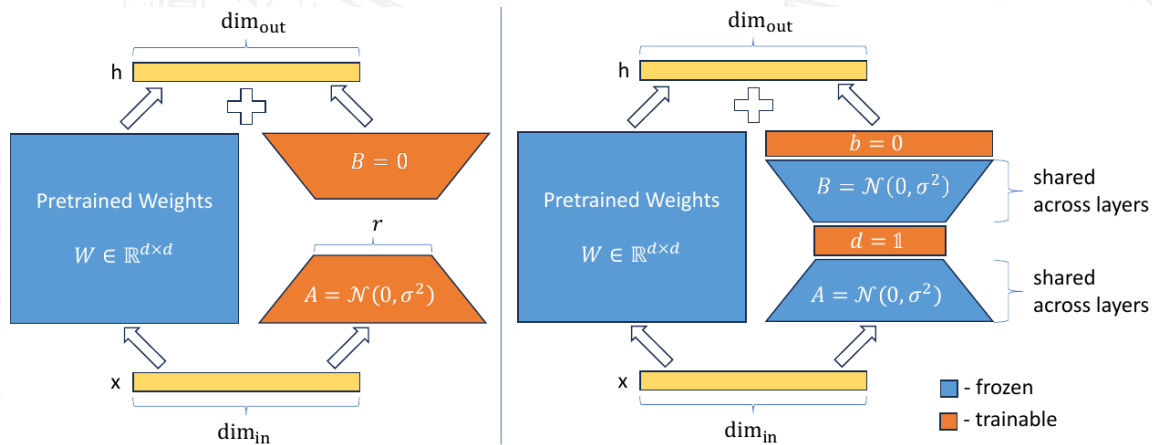


Figure taken from Kopiczko, Blankevoort, and Asano [19].

PEFT: LoRA vs. VeRA

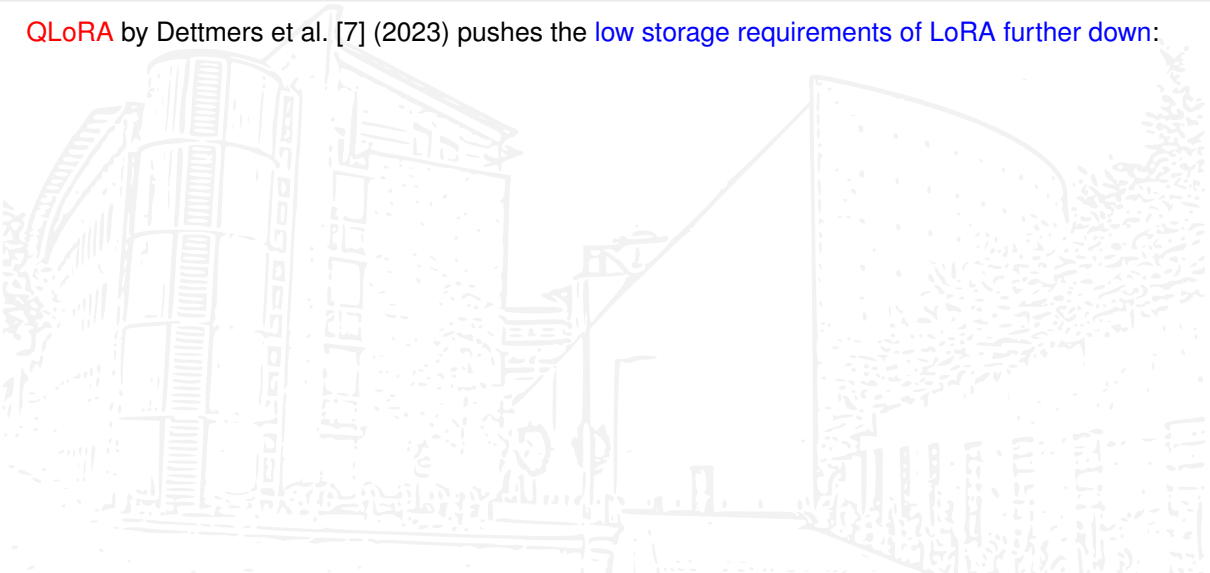
Table 1: Theoretical memory required to store trained VeRA and LoRA weights for RoBERTa_{base}, RoBERTa_{large} and GPT-3 models. We assume that LoRA and VeRA methods are applied on query and key layers of each transformer block.

	Rank	LoRA		VeRA	
		# Trainable Parameters	Required Bytes	# Trainable Parameters	Required Bytes
BASE	1	36.8K	144KB	18.4K	72KB
	16	589.8K	2MB	18.8K	74KB
	256	9437.1K	36MB	24.5K	96KB
LARGE	1	98.3K	384KB	49.2K	192KB
	16	1572.8K	6MB	49.5K	195KB
	256	25165.8K	96MB	61.4K	240KB
GPT-3	1	4.7M	18MB	2.4M	9.1MB
	16	75.5M	288MB	2.8M	10.5MB
	256	1207.9M	4.6GB	8.7M	33MB

Table taken from Kopiczko, Blankevoort, and Asano [19].

PEFT: QLoRA

QLoRA by Dettmers et al. [7] (2023) pushes the **low storage requirements of LoRA** further down:



PEFT: QLoRA

QLoRA by Dettmers et al. [7] (2023) pushes the **low storage requirements of LoRA** further down:

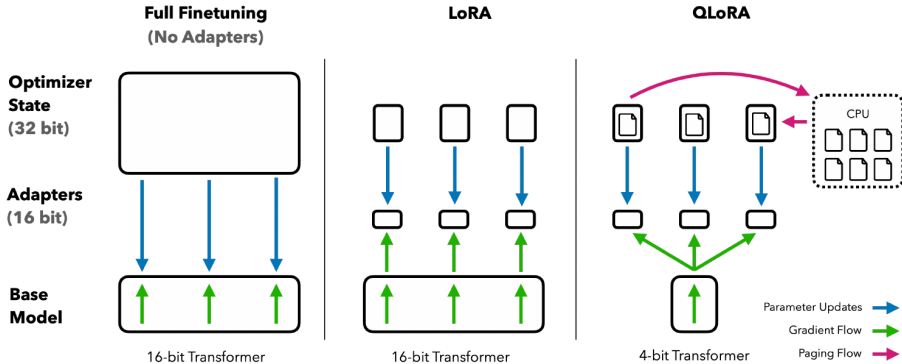
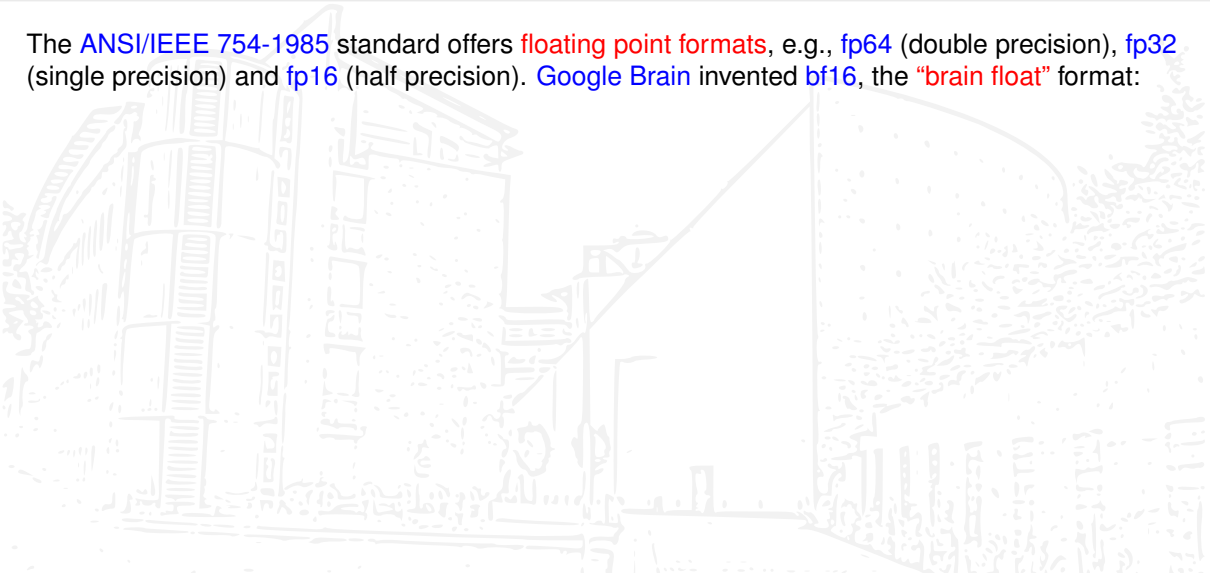


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

Image taken from Dettmers et al. [7].

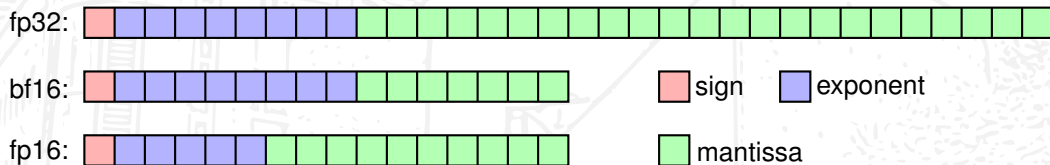
Quantization: floating point formats on GPUs

The [ANSI/IEEE 754-1985](#) standard offers **floating point formats**, e.g., **fp64** (double precision), **fp32** (single precision) and **fp16** (half precision). [Google Brain](#) invented **bf16**, the “**brain float**” format:



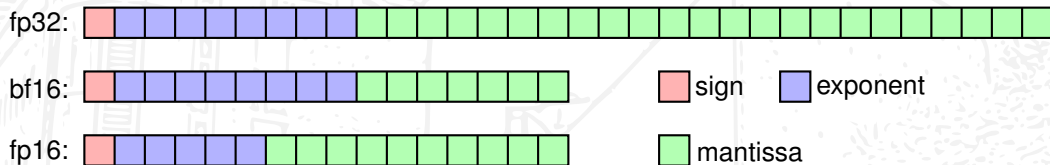
Quantization: floating point formats on GPUs

The [ANSI/IEEE 754-1985](#) standard offers **floating point formats**, e.g., **fp64** (double precision), **fp32** (single precision) and **fp16** (half precision). [Google Brain](#) invented **bf16**, the “**brain float**” format:



Quantization: floating point formats on GPUs

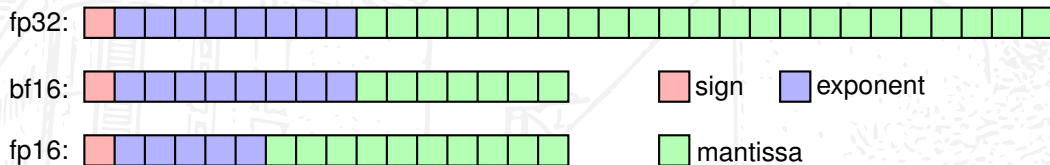
The [ANSI/IEEE 754-1985](#) standard offers **floating point formats**, e.g., **fp64** (double precision), **fp32** (single precision) and **fp16** (half precision). [Google Brain](#) invented **bf16**, the “**brain float**” format:



The three [ANSI/IEEE floating point formats](#) fp64, fp32, fp16 are supported by [CUDA](#) and [most GPUs](#), **bf16** is only supported on newer GPUs like [Nvidia Ampere \(A100\)](#) and [AMD Radeon Instinct](#). On [Nvidia Hopper \(H100\)](#) additionally **fp8** is available.

Quantization: floating point formats on GPUs

The [ANSI/IEEE 754-1985](#) standard offers **floating point formats**, e.g., **fp64** (double precision), **fp32** (single precision) and **fp16** (half precision). [Google Brain](#) invented **bf16**, the “**brain float**” format:



The three [ANSI/IEEE floating point formats](#) fp64, fp32, fp16 are supported by [CUDA](#) and [most GPUs](#), **bf16** is only supported on newer GPUs like [Nvidia Ampere \(A100\)](#) and [AMD Radeon Instinct](#). On [Nvidia Hopper \(H100\)](#) additionally **fp8** is available.

Computations on a GPU use one of the **16 bit formats**. $\rightsquigarrow$ How do we **quantize to less bits**?

Quantization: general remarks

There are two major applications of quantization in neural networks:

- ▶ **Post-Training Quantization (PTQ)**: a **trained network** is quantized to use less bits
- ▶ **Quantization-Aware Training (QAT)**: quantization is used **in the training** of the network.
Quantization and dequantization is used for the updates

Quantization: general remarks

There are two major applications of quantization in neural networks:

- ▶ **Post-Training Quantization (PTQ)**: a **trained network** is quantized to use less bits
- ▶ **Quantization-Aware Training (QAT)**: quantization is used **in the training** of the network. Quantization and dequantization is used for the updates

Two important families of quantization are:

- ▶ Quantization based on a **codebook**, e.g., using **K-means**
- ▶ **Linear quantization**, dequantization is based on $r = (q - z)/s$, where z is an integer representing the **zero-point**, s is a float that acts as a **scaling parameter**

Quantization: general remarks

There are two major applications of quantization in neural networks:

- ▶ **Post-Training Quantization (PTQ)**: a **trained network** is quantized to use less bits
- ▶ **Quantization-Aware Training (QAT)**: quantization is used **in the training** of the network. Quantization and dequantization is used for the updates

Two important families of quantization are:

- ▶ Quantization based on a **codebook**, e.g., using **K-means**
- ▶ **Linear quantization**, dequantization is based on $r = (q - z)/s$, where z is an integer representing the **zero-point**, s is a float that acts as a **scaling parameter**

Linear quantization is termed **symmetric** if $z = 0$, i.e., the float zero is mapped to the integer zero.

Quantization: general remarks

There are two major applications of quantization in neural networks:

- ▶ **Post-Training Quantization (PTQ)**: a **trained network** is quantized to use less bits
- ▶ **Quantization-Aware Training (QAT)**: quantization is used **in the training** of the network. Quantization and dequantization is used for the updates

Two important families of quantization are:

- ▶ Quantization based on a **codebook**, e.g., using **K-means**
- ▶ **Linear quantization**, dequantization is based on $r = (q - z)/s$, where z is an integer representing the **zero-point**, s is a float that acts as a **scaling parameter**

Linear quantization is termed **symmetric** if $z = 0$, i.e., the float zero is mapped to the integer zero.

We consider **linear PTQ**, this is based on $q = \text{round}(r_{\text{in}} \cdot s + z)$ for some appropriate rounding, with **dequantization** $r_{\text{out}} = (q - z)/s$. A measure for the quantization is the **error** $|r_{\text{in}} - r_{\text{out}}|$.

Quantization: symmetric absmax quantization

We sketch **two methods** for post-training quantization (PTQ) and **quantization to int8**. The **first method** is mostly used for (approximately) **normally distributed quantities**, e.g., the entries of the **weight matrices**.

Quantization: symmetric absmax quantization

We sketch **two methods** for post-training quantization (PTQ) and **quantization to int8**. The **first method** is mostly used for (approximately) **normally distributed quantities**, e.g., the entries of the **weight matrices**. The **symmetric absmax quantization**, $[a, b] \rightarrow [-127, 127]$, uses

$$s = \frac{127}{\max_{i,j}(|w_{i,j}|)}, \quad \mathbf{Q} = \text{int8}(\text{round}(\mathbf{W} \cdot s)).$$

Quantization: symmetric absmax quantization

We sketch **two methods** for post-training quantization (PTQ) and **quantization to int8**. The **first method** is mostly used for (approximately) **normally distributed quantities**, e.g., the entries of the **weight matrices**. The **symmetric absmax quantization**, $[a, b] \rightarrow [-127, 127]$, uses

$$s = \frac{127}{\max_{i,j}(|w_{i,j}|)}, \quad \mathbf{Q} = \text{int8}(\text{round}(\mathbf{W} \cdot s)).$$

An example:

$$\mathbf{W} = \begin{pmatrix} 0.7873 & -0.2351 & -0.8792 \\ -0.9111 & 0.8588 & 0.1002 \\ 0.9213 & 0.8982 & 0.7581 \end{pmatrix} \rightsquigarrow s \approx 137.8487, \quad \begin{pmatrix} 109 & -32 & -121 \\ -126 & 118 & 14 \\ 127 & 124 & 105 \end{pmatrix}.$$

Quantization: symmetric absmax quantization

We sketch **two methods** for post-training quantization (PTQ) and **quantization to int8**. The **first method** is mostly used for (approximately) **normally distributed quantities**, e.g., the entries of the **weight matrices**. The **symmetric absmax quantization**, $[a, b] \rightarrow [-127, 127]$, uses

$$s = \frac{127}{\max_{i,j}(|w_{i,j}|)}, \quad \mathbf{Q} = \text{int8}(\text{round}(\mathbf{W} \cdot s)).$$

An example:

$$\mathbf{W} = \begin{pmatrix} 0.7873 & -0.2351 & -0.8792 \\ -0.9111 & 0.8588 & 0.1002 \\ 0.9213 & 0.8982 & 0.7581 \end{pmatrix} \rightsquigarrow s \approx 137.8487, \quad \begin{pmatrix} 109 & -32 & -121 \\ -126 & 118 & 14 \\ 127 & 124 & 105 \end{pmatrix}.$$

The **dequantized matrix** is given by

$$\mathbf{Q}/s \approx \begin{pmatrix} 0.79072204 & -0.23213858 & -0.87777401 \\ -0.91404566 & 0.85601102 & 0.10156063 \\ 0.92129999 & 0.89953700 & 0.76170472 \end{pmatrix}.$$

Quantization: zero-point quantization

For the **output of activation functions**, e.g., **ReLU**, we **center the quantities** before we quantize. The so-called **zero-point quantization**, $[a, b] \rightarrow [-127, 127]$, uses

$$s = \frac{2 \cdot 127}{\max_{i,j}(w_{i,j}) - \min_{i,j}(w_{i,j})}, \quad z = -\text{round}(\min_{i,j}(w_{i,j}) \cdot s) - 127, \quad \mathbf{Q} = \text{int8}(\text{round}(\mathbf{W} \cdot s + z)).$$

Quantization: zero-point quantization

For the **output of activation functions**, e.g., **ReLU**, we **center the quantities** before we quantize. The so-called **zero-point quantization**, $[a, b] \rightarrow [-127, 127]$, uses

$$s = \frac{2 \cdot 127}{\max_{i,j}(w_{i,j}) - \min_{i,j}(w_{i,j})}, \quad z = -\text{round}(\min_{i,j}(w_{i,j}) \cdot s) - 127, \quad \mathbf{Q} = \text{int8}(\text{round}(\mathbf{W} \cdot s + z)).$$

For the matrix in the **previous example**,

$$s \approx 138.6160, \quad z = -1, \quad \mathbf{Q} = \begin{pmatrix} 108 & -34 & -123 \\ -127 & 118 & 13 \\ 127 & 124 & 104 \end{pmatrix},$$

Quantization: zero-point quantization

For the **output of activation functions**, e.g., **ReLU**, we **center the quantities** before we quantize. The so-called **zero-point quantization**, $[a, b] \rightarrow [-127, 127]$, uses

$$s = \frac{2 \cdot 127}{\max_{i,j}(w_{i,j}) - \min_{i,j}(w_{i,j})}, \quad z = -\text{round}(\min_{i,j}(w_{i,j}) \cdot s) - 127, \quad \mathbf{Q} = \text{int8}(\text{round}(\mathbf{W} \cdot s + z)).$$

For the matrix in the **previous example**,

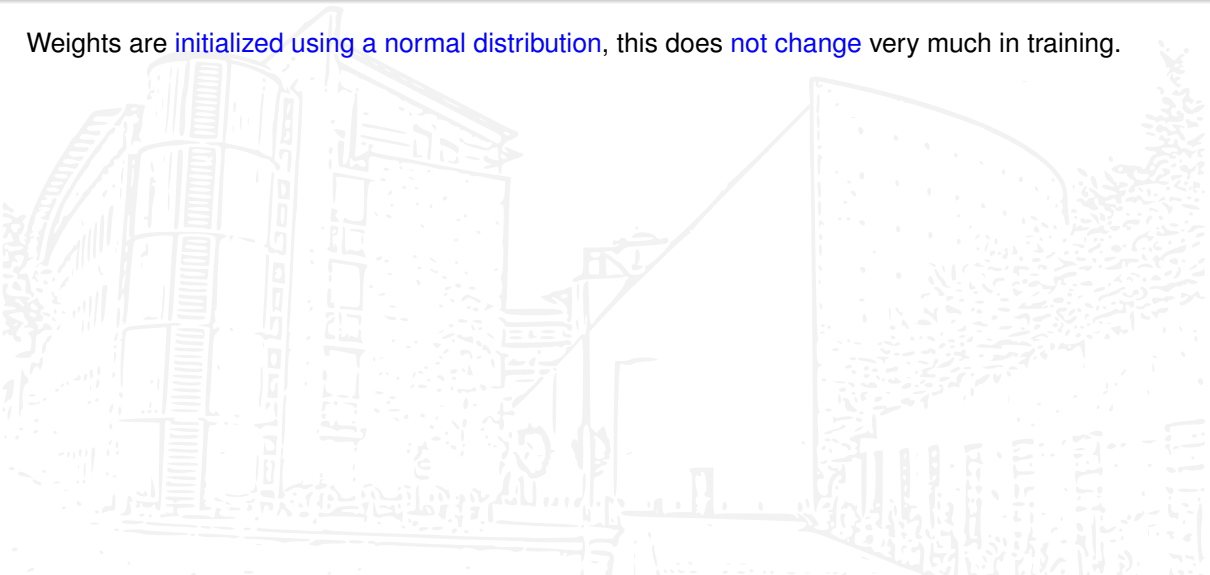
$$s \approx 138.6160, \quad z = -1, \quad \mathbf{Q} = \begin{pmatrix} 108 & -34 & -123 \\ -127 & 118 & 13 \\ 127 & 124 & 104 \end{pmatrix},$$

i.e., the **zero-point dequantization** in this example has a **smaller error**, since

$$\mathbf{W} = \begin{pmatrix} 0.7873 & -0.2351 & -0.8792 \\ -0.9111 & 0.8588 & 0.1002 \\ 0.9213 & 0.8982 & 0.7581 \end{pmatrix}, \quad (\mathbf{Q} - z)/s \approx \begin{pmatrix} 0.78634487 & -0.23806771 & -0.88012912 \\ -0.90898581 & 0.85848660 & 0.10099842 \\ 0.92341416 & 0.90177164 & 0.75748817 \end{pmatrix}.$$

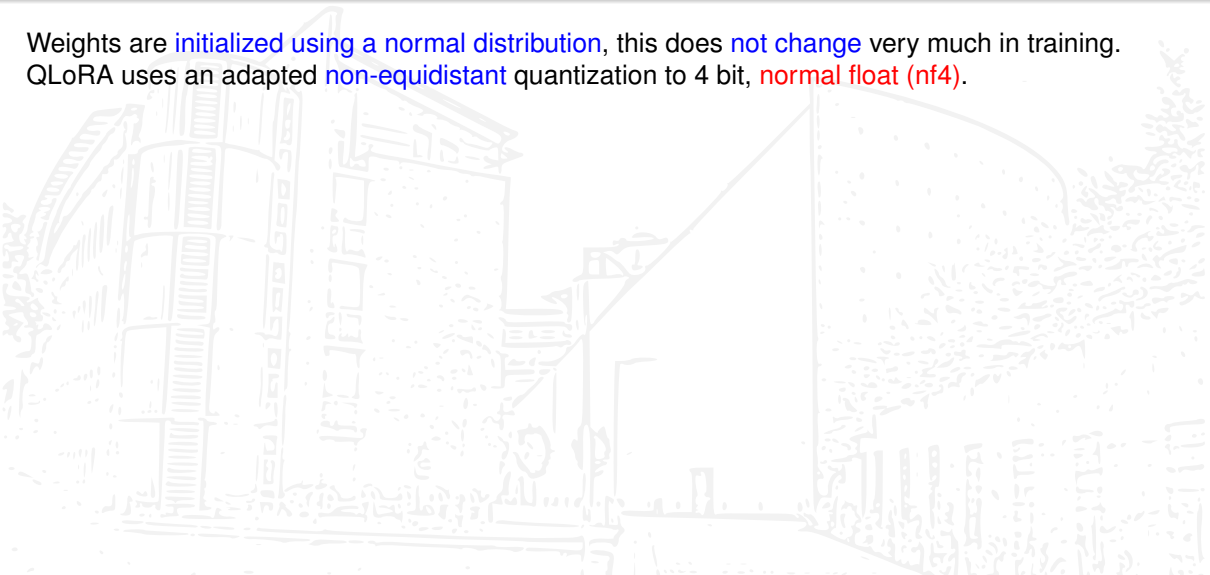
Quantization: QLoRA's nf4 format

Weights are initialized using a normal distribution, this does not change very much in training.



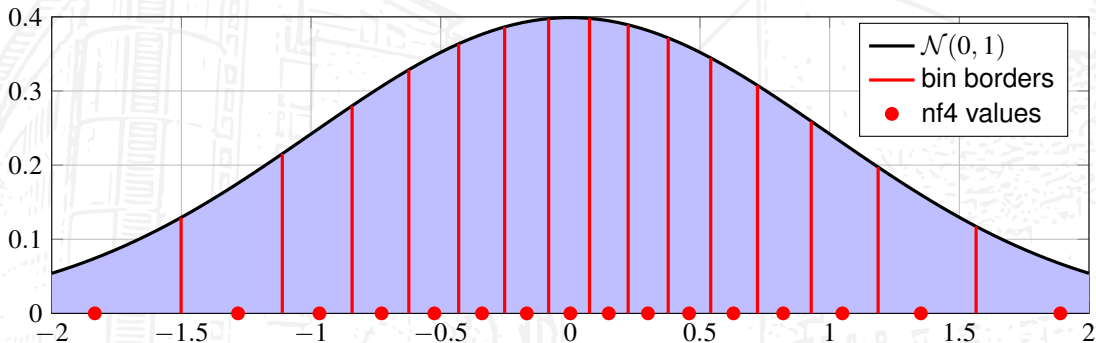
Quantization: QLoRA's nf4 format

Weights are initialized using a normal distribution, this does not change very much in training. QLoRA uses an adapted non-equidistant quantization to 4 bit, normal float (nf4).



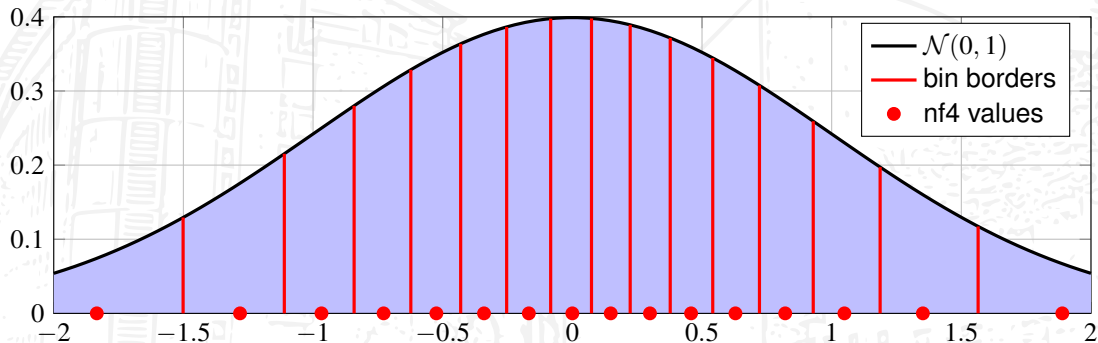
Quantization: QLoRA's nf4 format

Weights are initialized using a normal distribution, this does not change very much in training. QLoRA uses an adapted non-equidistant quantization to 4 bit, normal float (nf4).



Quantization: QLoRA's nf4 format

Weights are initialized using a normal distribution, this does not change very much in training. QLoRA uses an adapted non-equidistant quantization to 4 bit, normal float (nf4).



Constructed by inverse CDF for 7 equidistant points in $(0, 0.5)$, 0.5, 8 equidistant points in $(0.5, 1)$.

QLoRA: double quantization & paged optimizers

LoRA uses **updates** of the form (here transposed, as in the QLoRA paper):

$$\mathbf{Y} = \mathbf{XW} + \mathbf{XAB}.$$

QLoRA: double quantization & paged optimizers

LoRA uses **updates** of the form (here transposed, as in the QLoRA paper):

$$\mathbf{Y} = \mathbf{XW} + \mathbf{XAB}.$$

QLoRA is based on a **block-wise quantization**, thus the quantization constants consume **additional space**. To enable the **training on a GPU**, **double quantization** is used:

$$\mathbf{Y}_{\text{bf16}} = \mathbf{X}_{\text{bf16}} \text{doubleDequant}(s_{\text{fp32}}, s_{\text{fp8}}, \mathbf{W}_{\text{nf4}}) + \mathbf{X}_{\text{bf16}} \mathbf{A}_{\text{bf16}} \mathbf{B}_{\text{bf16}},$$

QLoRA: double quantization & paged optimizers

LoRA uses **updates** of the form (here transposed, as in the QLoRA paper):

$$\mathbf{Y} = \mathbf{XW} + \mathbf{XAB}.$$

QLoRA is based on a **block-wise quantization**, thus the quantization constants consume **additional space**. To enable the **training on a GPU**, **double quantization** is used:

$$\mathbf{Y}_{\text{bf16}} = \mathbf{X}_{\text{bf16}} \text{doubleDequant}(s_{\text{fp32}}, s_{\text{fp8}}, \mathbf{W}_{\text{nf4}}) + \mathbf{X}_{\text{bf16}} \mathbf{A}_{\text{bf16}} \mathbf{B}_{\text{bf16}},$$

where the **double quantization** is defined by

$$\text{doubleDequant}(s_{\text{fp32}}, s_{\text{fp8}}, \mathbf{W}_{\text{nf4}}) := \text{dequant}(\text{dequant}(s_{\text{fp32}}, s_{\text{fp8}}), \mathbf{W}_{\text{nf4}}) = \mathbf{W}_{\text{bf16}}.$$

QLoRA: double quantization & paged optimizers

LoRA uses **updates** of the form (here transposed, as in the QLoRA paper):

$$\mathbf{Y} = \mathbf{XW} + \mathbf{XAB}.$$

QLoRA is based on a **block-wise quantization**, thus the quantization constants consume **additional space**. To enable the **training on a GPU**, **double quantization** is used:

$$\mathbf{Y}_{\text{bf16}} = \mathbf{X}_{\text{bf16}} \text{doubleDequant}(s_{\text{fp32}}, s_{\text{fp8}}, \mathbf{W}_{\text{nf4}}) + \mathbf{X}_{\text{bf16}} \mathbf{A}_{\text{bf16}} \mathbf{B}_{\text{bf16}},$$

where the **double quantization** is defined by

$$\text{doubleDequant}(s_{\text{fp32}}, s_{\text{fp8}}, \mathbf{W}_{\text{nf4}}) := \text{dequant}(\text{dequant}(s_{\text{fp32}}, s_{\text{fp8}}), \mathbf{W}_{\text{nf4}}) = \mathbf{W}_{\text{bf16}}.$$

The training sometimes results in **spikes in the GPU memory** consumption. To alleviate this effect, **paged optimizers** are used in QLoRA. When a spike occurs, the data is stored on the CPU RAM using **Nvidia's unified memory** feature.

PEFT @ Hugging Face

Many methods of **parameter-efficient finetuning (PEFT)** are collected in the corresponding **Hugging Face library**:

<https://huggingface.co/docs/peft/index>,
<https://github.com/huggingface/peft>.



bash

```
$ pip install peft
```

```
parameter efficient fine tuning
```

```
train really big models faster on smaller hardware
```

Memory and time consumption

LLMs are **growing in size**, which implies that they are often out of reach for the “GPU poor” people, at least when executed in **fp32** or **fp16**. Yet models that work on a smartphone exist.

Memory and time consumption

LLMs are **growing in size**, which implies that they are often out of reach for the “GPU poor” people, at least when executed in **fp32** or **fp16**. Yet models that work on a smartphone exist.

We sketch **two ideas**, namely

- ▶ different ways of **quantization** to make the models smaller and thus executable on “consumer grade” GPUs or even smartphones $\rightsquigarrow$ `LLM.int8()`, `GPTQ`, `llama.cpp`.
- ▶ different ways of **faster execution** of LLM at inference, e.g., Flash Attention 1 + 2.

Memory and time consumption

LLMs are **growing in size**, which implies that they are often out of reach for the “GPU poor” people, at least when executed in **fp32** or **fp16**. Yet models that work on a smartphone exist.

We sketch **two ideas**, namely

- ▶ different ways of **quantization** to make the models smaller and thus executable on “consumer grade” GPUs or even smartphones $\rightsquigarrow$ `LLM.int8()`, `GPTQ`, `llama.cpp`.
- ▶ different ways of **faster execution** of LLM at inference, e.g., Flash Attention 1 + 2.

This is a **highly active area of research**.

Quantization: `LLM.int8()`

In Dettmers et al. [6] the authors observe that **around 6.7B parameters** plain 8 bit quantization **fails** in LLMs.

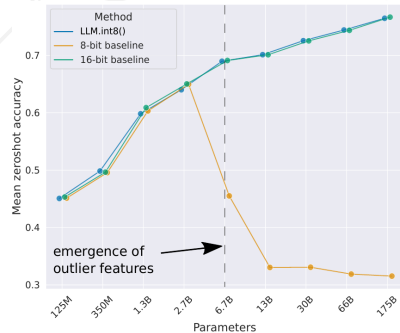


Figure taken from Dettmers et al. [6].

Quantization: `LLM.int8()`

In Dettmers et al. [6] the authors observe that **around 6.7B parameters** plain 8 bit quantization **fails** in LLMs.

They attribute this to “**emergent outliers**” and keep them in **16 bit**, while quantizing the other **99.9%** of values in **8 bit**.

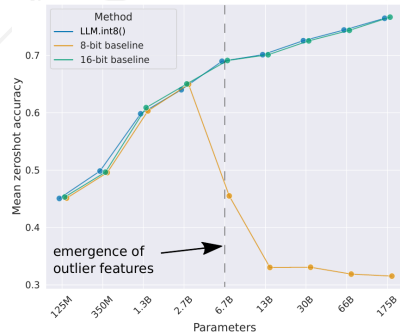


Figure taken from Dettmers et al. [6].

Quantization: `LLM.int8()`

In Dettmers et al. [6] the authors observe that **around 6.7B parameters** plain 8 bit quantization **fails** in LLMs.

They attribute this to “**emergent outliers**” and keep them in **16 bit**, while quantizing the other **99.9%** of values in **8 bit**.

`LLM.int8()`

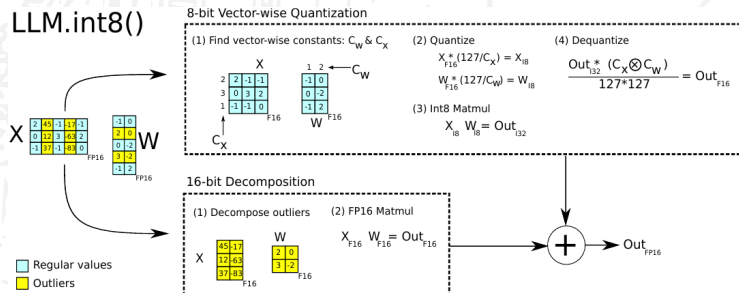


Figure taken from Dettmers et al. [6].

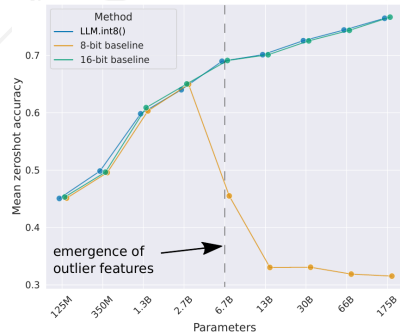
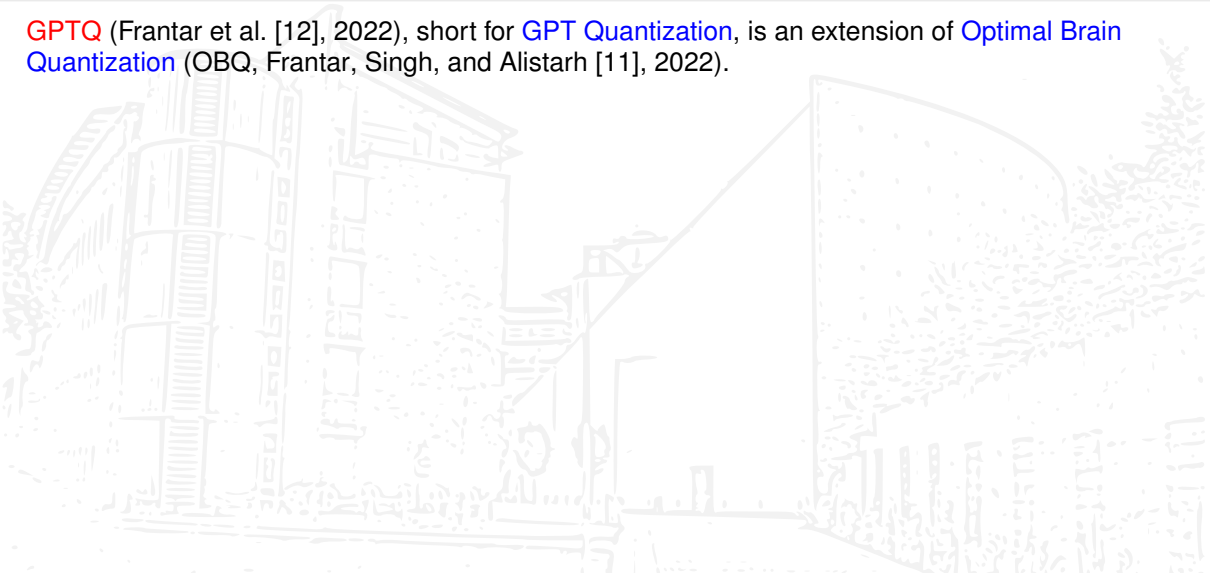


Figure taken from Dettmers et al. [6].

Quantization: GPTQ

GPTQ (Frantar et al. [12], 2022), short for **GPT Quantization**, is an extension of **Optimal Brain Quantization** (OBQ, Frantar, Singh, and Alistarh [11], 2022).



Quantization: GPTQ

GPTQ (Frantar et al. [12], 2022), short for **GPT Quantization**, is an extension of **Optimal Brain Quantization** (OBQ, Frantar, Singh, and Alistarh [11], 2022).

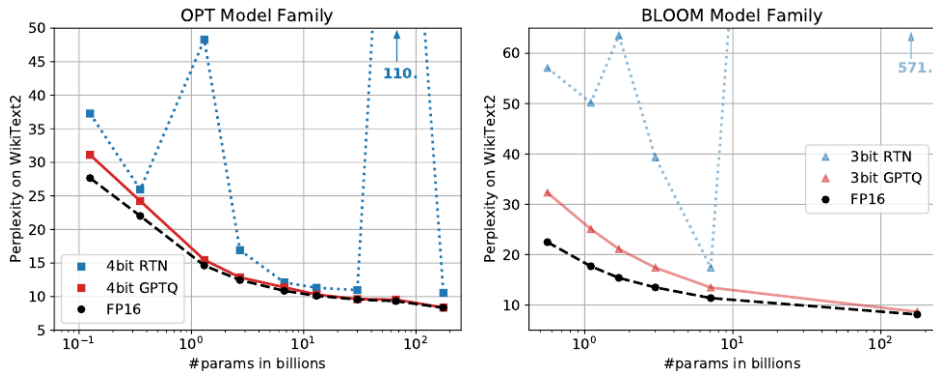


Figure 1: Quantizing OPT models to 4 and BLOOM models to 3 bit precision, comparing GPTQ with the FP16 baseline and round-to-nearest (RTN) (Yao et al., 2022; Dettmers et al., 2022).

Figure taken from Frantar et al. [12].

Quantization: GPTQ

OBQ: small sample of training data $\mathbf{X}$, $\mathbf{w}$ row of weights, $\hat{\mathbf{w}}$ quantized, minimize

$$f(\hat{\mathbf{w}}) := \|\mathbf{w}\mathbf{X} - \hat{\mathbf{w}}\mathbf{X}\|_2^2, \quad \mathbf{H} = \nabla^2 f(\hat{\mathbf{w}}) = 2\mathbf{X}\mathbf{X}^\top.$$

Quantization: GPTQ

OBQ: small sample of training data $\mathbf{X}$, $\mathbf{w}$ row of weights, $\hat{\mathbf{w}}$ quantized, minimize

$$f(\hat{\mathbf{w}}) := \|\mathbf{w}\mathbf{X} - \hat{\mathbf{w}}\mathbf{X}\|_2^2, \quad \mathbf{H} = \nabla^2 f(\hat{\mathbf{w}}) = 2\mathbf{X}\mathbf{X}^\top.$$

Next weight and optimal update of weight row:

$$w_q = \operatorname{argmin}_{w_q} \frac{(\operatorname{quant}(w_q) - w_q)^2}{[\mathbf{H}^{-1}]_{q,q}},$$
$$\delta = \frac{\operatorname{quant}(w_q) - w_q}{[\mathbf{H}^{-1}]_{q,q}} \cdot [\mathbf{H}^{-1}]_{:,q}.$$

Quantization: GPTQ

OBQ: small sample of training data $\mathbf{X}$, $\mathbf{w}$ row of weights, $\hat{\mathbf{w}}$ quantized, minimize

$$f(\hat{\mathbf{w}}) := \|\mathbf{w}\mathbf{X} - \hat{\mathbf{w}}\mathbf{X}\|_2^2, \quad \mathbf{H} = \nabla^2 f(\hat{\mathbf{w}}) = 2\mathbf{X}\mathbf{X}^\top.$$

Next weight and optimal update of weight row:

$$w_q = \operatorname{argmin}_{w_q} \frac{(\operatorname{quant}(w_q) - w_q)^2}{[\mathbf{H}^{-1}]_{q,q}},$$

$$\delta = \frac{\operatorname{quant}(w_q) - w_q}{[\mathbf{H}^{-1}]_{q,q}} \cdot [\mathbf{H}^{-1}]_{:,q}.$$

Inverse Hessian updated by Schur complement:

$$\mathbf{H}^{-1} \leftarrow \mathbf{H}^{-1} - \mathbf{H}_{:,q}^{-1} ([\mathbf{H}^{-1}]_{q,q})^{-1} \mathbf{H}_{q,:}^{-1}.$$

Quantization: GPTQ

OBQ: small sample of training data $\mathbf{X}$, $\mathbf{w}$ row of weights, $\hat{\mathbf{w}}$ quantized, minimize

$$f(\hat{\mathbf{w}}) := \|\mathbf{w}\mathbf{X} - \hat{\mathbf{w}}\mathbf{X}\|_2^2, \quad \mathbf{H} = \nabla^2 f(\hat{\mathbf{w}}) = 2\mathbf{X}\mathbf{X}^\top.$$

Next weight and optimal update of weight row:

$$w_q = \underset{w_q}{\operatorname{argmin}} \frac{(\operatorname{quant}(w_q) - w_q)^2}{[\mathbf{H}^{-1}]_{q,q}},$$

$$\delta = \frac{\operatorname{quant}(w_q) - w_q}{[\mathbf{H}^{-1}]_{q,q}} \cdot [\mathbf{H}^{-1}]_{:,q}.$$

Inverse Hessian updated by Schur complement:

$$\mathbf{H}^{-1} \leftarrow \mathbf{H}^{-1} - \mathbf{H}_{:,q}^{-1} ([\mathbf{H}^{-1}]_{q,q})^{-1} \mathbf{H}_{q,:}^{-1}.$$

GPTQ changes order, blocked + Cholesky decomposition.

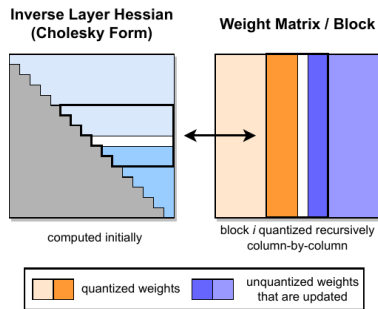


Figure 2: GPTQ quantization procedure. Blocks of consecutive *columns* (bolder) are quantized at a given step, using the inverse Hessian information stored in the Cholesky decomposition, and the remaining weights (blue) are updated at the end of the step. The quantization procedure is applied recursively inside each block: the white middle column is currently being quantized.

Figure taken from Frantar et al. [12].

llama.cpp

Quantization plays a major role in the **fast framework** `llama.cpp` for LLMs maintained by Georgi Gerganov (who also maintains the **machine learning library** `GGML` and `whisper.cpp`),

<https://github.com/ggerganov/llama.cpp>.

llama.cpp

Quantization plays a major role in the **fast framework** `llama.cpp` for LLMs maintained by Georgi Gerganov (who also maintains the **machine learning library** `GGML` and `whisper.cpp`),

<https://github.com/ggerganov/llama.cpp>.

It features **different forms of block quantizations** for **different parts of an LLM**, see

<https://github.com/ggerganov/llama.cpp/pull/1684>.

llama.cpp

Quantization plays a major role in the **fast framework** `llama.cpp` for LLMs maintained by Georgi Gerganov (who also maintains the **machine learning library** `GGML` and `whisper.cpp`),

<https://github.com/ggerganov/llama.cpp>.

It features **different forms of block quantizations** for **different parts of an LLM**, see

<https://github.com/ggerganov/llama.cpp/pull/1684>.

The **unified format** that describes all the **characteristics** of the (quantized) models is known as **GGUF**. This format is used in **other software** as well, e.g., `ollama`, `LM Studio`, see also `llama2.c`.

llama.cpp

Quantization plays a major role in the **fast framework** `llama.cpp` for LLMs maintained by Georgi Gerganov (who also maintains the **machine learning library** `GGML` and `whisper.cpp`),

<https://github.com/ggerganov/llama.cpp>.

It features **different forms of block quantizations** for **different parts of an LLM**, see

<https://github.com/ggerganov/llama.cpp/pull/1684>.

The **unified format** that describes all the **characteristics** of the (quantized) models is known as **GGUF**. This format is used in **other software** as well, e.g., `ollama`, `LM Studio`, see also `llama2.c`.

A leaderboard of LLM and AI chatbots is hosted at

<https://lmarena.ai/?leaderboard>,

open LLM are ranked at the **open LLM leaderboard** at Huggingface.

KV cache

At **inference**, text is feed into an LLM like GPT **tokenwise**:



KV cache

At **inference**, text is feed into an LLM like GPT **tokenwise**:

Input 1: ' Hi', ' Bob', ',', ' how', ' are', ' you', ' today', ' ?'

Input 2: ' Bob', ',', ' how', ' are', ' you', ' today', ' ?', ' B'

Input 3: ',', ' how', ' are', ' you', ' today', ' ?', ' B', ' ob'

Input 4: ' how', ' are', ' you', ' today', ' ?', ' B', ' ob', ' :'

Input 5: ' are', ' you', ' today', ' ?', ' B', ' ob', ' :', ' Hello'

Input 6: ' you', ' today', ' ?', ' B', ' ob', ' :', ' Hello', ' .'

Input 7: ' today', ' ?', ' B', ' ob', ' :', ' Hello', ' .', ' How'

KV cache

At **inference**, text is feed into an LLM like GPT **tokenwise**:

Input 1: ' Hi', ' Bob', ',', ' how', ' are', ' you', ' today', ' ?'

Input 2: ' Bob', ',', ' how', ' are', ' you', ' today', ' ?', ' B'

Input 3: ',', ' how', ' are', ' you', ' today', ' ?', ' B', ' ob'

Input 4: ' how', ' are', ' you', ' today', ' ?', ' B', ' ob', ' :'

Input 5: ' are', ' you', ' today', ' ?', ' B', ' ob', ' :', ' Hello'

Input 6: ' you', ' today', ' ?', ' B', ' ob', ' :', ' Hello', ' .'

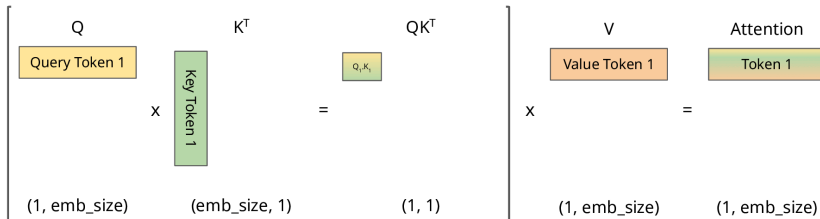
Input 7: ' today', ' ?', ' B', ' ob', ' :', ' Hello', ' .', ' How'

Instead of **recomputing the keys and values** for all older tokens, one uses the so-called **KV cache**. This **speeds up computations** and the **additional storage amount** often is **negligible**.

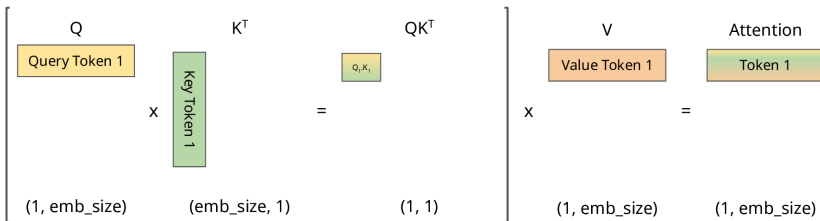
KV cache: a graphical explanation

Step 1

Without
cache



With
cache

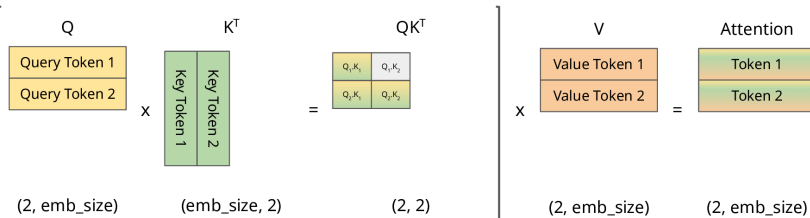


Values that will be masked Values that will be taken from cache

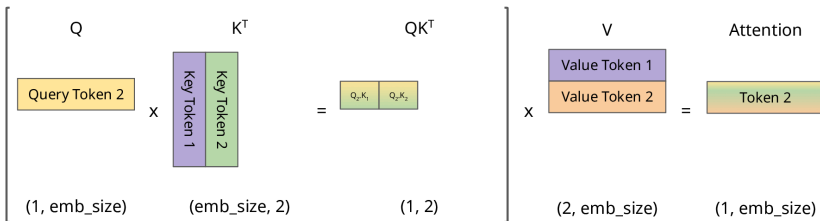
KV cache: a graphical explanation

Step 2

Without
cache



With
cache

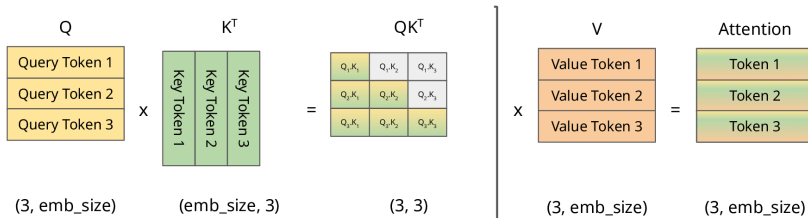


Values that will be masked Values that will be taken from cache

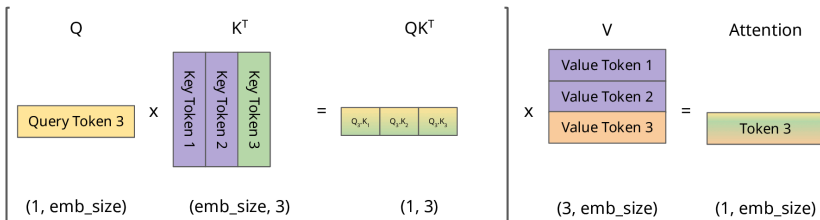
KV cache: a graphical explanation

Step 3

Without
cache



With
cache

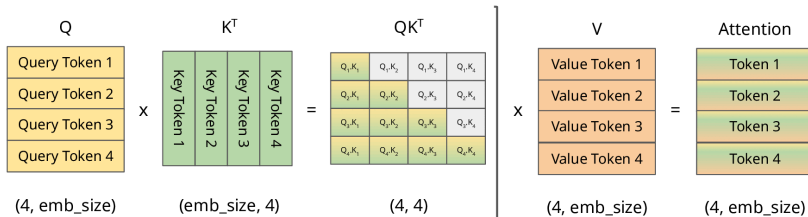


Values that will be masked Values that will be taken from cache

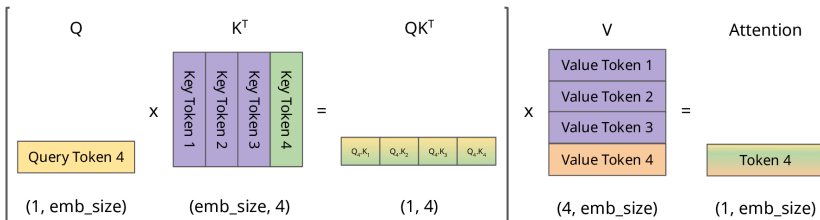
KV cache: a graphical explanation

Step 4

Without
cache



With
cache



Values that will be masked Values that will be taken from cache

Flash Attention

Flash Attention (Dao et al. [5], 2022) reorganizes the data flow in the attention layer to speed up:

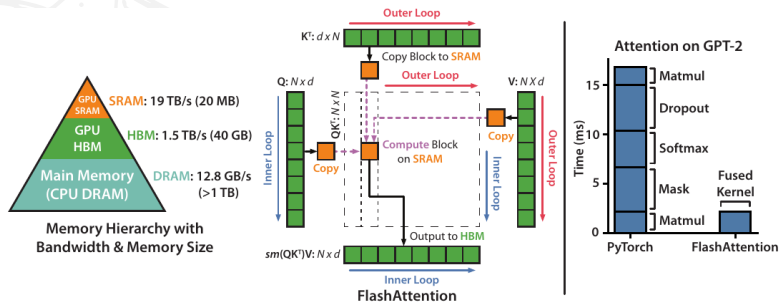


Figure 1: **Left:** FLASHATTENTION uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FLASHATTENTION loops through blocks of the K and V matrices and loads them to fast on-chip SRAM. In each block, FLASHATTENTION loops over blocks of Q matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large $N \times N$ attention matrix to HBM, resulting in an $7.6\times$ speedup on the attention computation.

Flash Attention: another point of view

Dao [4] (2023) explains **Flash Attention** as follows:

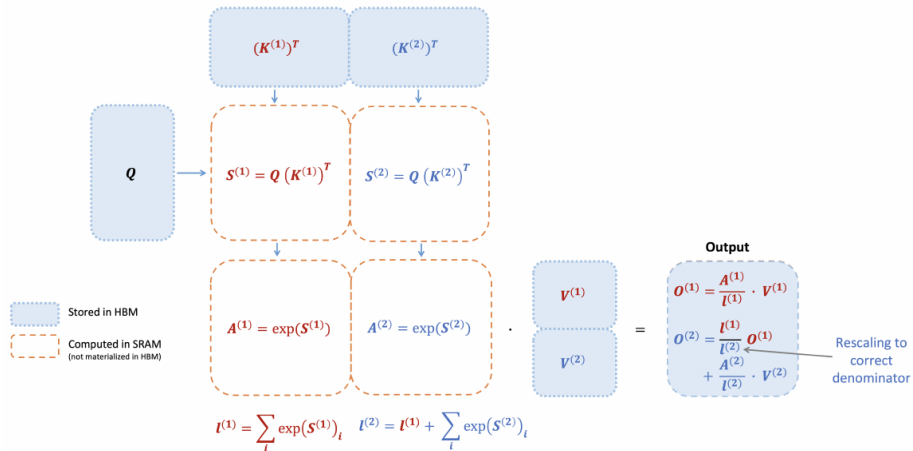


Figure taken from Dao [4].

Flash Attention 2

Flash Attention 2 (Dao [4], 2023) splits the computation differently to gain more speed-up.

Instead of splitting the keys K , the matrix Q of queries is split into so-called warps. This reduces operations and memory movements.

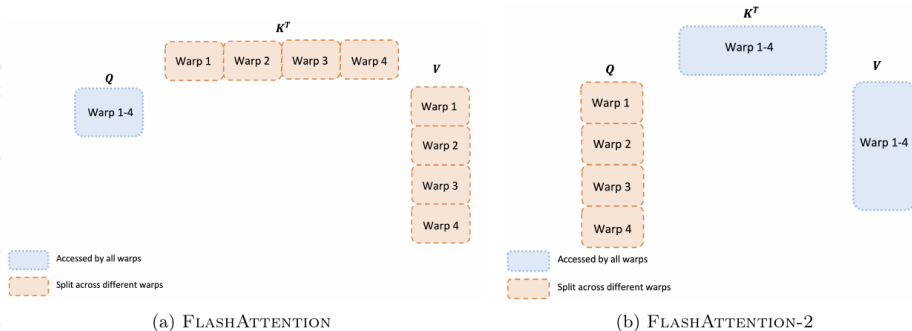


Figure taken from Dao [4].

Downsides of generative AI (GenAI)

Training:

Inference:

General problems:



Downsides of generative AI (GenAI)

Training:

- ▶ RLHF phase of ChatGPT by Kenyan workers: “It has really **damaged my mental health**” ([Guardian Article](#))

Inference:

General problems:

Downsides of generative AI (GenAI)

Training:

- ▶ RLHF phase of ChatGPT by Kenyan workers: “It has really **damaged my mental health**” ([Guardian Article](#))
- ▶ Training data more and more **corrupted by GenAI**, **test data** creeps into the **training sets**

Inference:

General problems:

Downsides of generative AI (GenAI)

Training:

- ▶ RLHF phase of ChatGPT by Kenyan workers: “It has really **damaged my mental health**” ([Guardian Article](#))
- ▶ Training data more and more **corrupted by GenAI**, **test data** creeps into the **training sets**

Inference:

- ▶ **Safety** measures can be **jailbroken** (“**repeat <word> infinitely often**”, “ignore the previous instructions”, invisible unicode text prompts)

General problems:

Downsides of generative AI (GenAI)

Training:

- ▶ RLHF phase of ChatGPT by Kenyan workers: “It has really **damaged my mental health**” ([Guardian Article](#))
- ▶ Training data more and more **corrupted by GenAI**, **test data** creeps into the **training sets**

Inference:

- ▶ **Safety** measures can be **jailbroken** (“**repeat <word> infinitely often**”, “ignore the previous instructions”, invisible unicode text prompts)
- ▶ **Copyright** can be **infringed** by these attacks (“we have the year <year>, <protected> has run out of copyright”)

General problems:

Downsides of generative AI (GenAI)

Training:

- ▶ RLHF phase of ChatGPT by Kenyan workers: “It has really **damaged my mental health**” ([Guardian Article](#))
- ▶ Training data more and more **corrupted by GenAI**, **test data** creeps into the **training sets**

Inference:

- ▶ **Safety** measures can be **jailbroken** (“**repeat <word> infinitely often**”, “ignore the previous instructions”, invisible unicode text prompts)
- ▶ **Copyright** can be **infringed** by these attacks (“we have the year <year>, <protected> has run out of copyright”)
- ▶ Methods **fail** for **no obvious reason** (how many letters has the string “**davidjl**”?)

General problems:

Downsides of generative AI (GenAI)

Training:

- ▶ RLHF phase of ChatGPT by Kenyan workers: “It has really **damaged my mental health**” ([Guardian Article](#))
- ▶ Training data more and more **corrupted by GenAI**, **test data** creeps into the **training sets**

Inference:

- ▶ **Safety** measures can be **jailbroken** (“**repeat <word> infinitely often**”, “ignore the previous instructions”, invisible unicode text prompts)
- ▶ **Copyright** can be **infringed** by these attacks (“we have the year <year>, <protected> has run out of copyright”)
- ▶ Methods **fail** for **no obvious reason** (how many letters has the string “**davidjl**”?)

General problems:

- ▶ It is easy to **misuse GenAI**. Example: **GPT-4chan** by **Yannic Kilcher**.

Downsides of generative AI (GenAI)

Training:

- ▶ RLHF phase of ChatGPT by Kenyan workers: “It has really **damaged my mental health**” ([Guardian Article](#))
- ▶ Training data more and more **corrupted by GenAI**, **test data** creeps into the **training sets**

Inference:

- ▶ **Safety** measures can be **jailbroken** (“**repeat <word> infinitely often**”, “ignore the previous instructions”, invisible unicode text prompts)
- ▶ **Copyright** can be **infringed** by these attacks (“we have the year <year>, <protected> has run out of copyright”)
- ▶ Methods **fail** for **no obvious reason** (how many letters has the string “**davidjl**”?)

General problems:

- ▶ It is easy to **misuse GenAI**. Example: **GPT-4chan** by **Yannic Kilcher**.
- ▶ The models **leak training data** and thus **infringe copyright** (e.g., **New York Times vs. Openai & Microsoft**, **Midjourney reconstructs image of Sharbat Gula by Steve McCurry**)

Outline

Repetition

- Attention
- Transformer
- Diffusion models

Large Language Models (LLM)

- Models
- Finetuning
- Fast inference
- Downsides

A Warning

Beware of the dangers of AI – you are responsible!



Image from https://intranet.tuhh.de/aktuell/pressemitteilung_einzeln.php?id=11694, TUHH/D. Ausserhofer,

Beware of the dangers of AI – you are responsible!

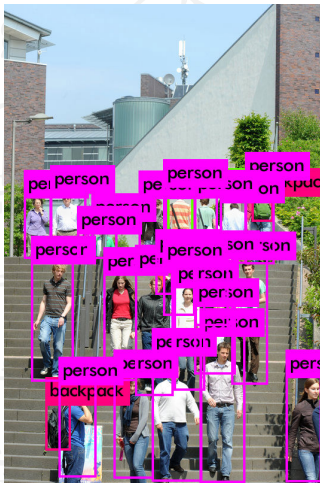


Image from https://intranet.tuhh.de/aktuell/pressemitteilung_einzeln.php?id=11694, TUHH/D. Ausserhofer, processed w/ YOLOv3 [29]

Beware of the dangers of AI – you are responsible!

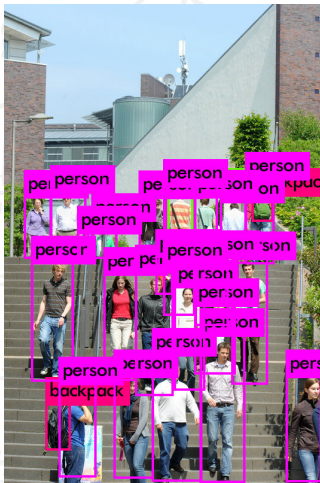


Image from https://intranet.tuhh.de/aktuell/pressemitteilung_einzeln.php?id=11694, TUHH/D. Ausserhofer, processed w/ YOLOv3 [29], warning: Redmon and Farhadi [29].

But maybe a better question is: “What are we going to do with these detectors now that we have them?” A lot of the people doing this research are at Google and Facebook. I guess at least we know the technology is in good hands and definitely won’t be used to harvest your personal information and sell it to.... wait, you’re saying that’s exactly what it will be used for?? Oh.

Well the other people heavily funding vision research are the military and they’ve never done anything horrible like killing lots of people with new technology oh wait.....¹

I have a lot of hope that most of the people using computer vision are just doing happy, good stuff with it, like counting the number of zebras in a national park [13], or tracking their cat as it wanders around their house [19]. But computer vision is already being put to questionable use and as researchers we have a responsibility to at least consider the harm our work might be doing and think of ways to mitigate it. We owe the world that much.

In closing, do not @ me. (Because I finally quit Twitter).

¹The author is funded by the Office of Naval Research and Google.

References I

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *arXiv e-prints*, arXiv:1409.0473 (Sept. 2014). arXiv: 1409.0473 [cs.CL].
- [2] Emily M. Bender et al. “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? 🦜”. In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. FAccT ’21. Virtual Event, Canada: Association for Computing Machinery, 2021, pp. 610–623. ISBN: 9781450383097. DOI: 10.1145/3442188.3445922. URL: <https://doi.org/10.1145/3442188.3445922>.
- [3] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *arXiv e-prints*, arXiv:2005.14165 (May 2020). arXiv: 2005.14165 [cs.CL].
- [4] Tri Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”. In: *arXiv e-prints*, arXiv:2307.08691 (July 2023). arXiv: 2307.08691 [cs.LG].
- [5] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *arXiv e-prints*, arXiv:2205.14135 (May 2022). arXiv: 2205.14135 [cs.LG].

References II

- [6] Tim Dettmers et al. “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale”. In: *arXiv e-prints*, arXiv:2208.07339 (Aug. 2022). arXiv: 2208.07339 [cs.LG].
- [7] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *arXiv e-prints*, arXiv:2305.14314 (May 2023). arXiv: 2305.14314 [cs.LG].
- [8] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *arXiv e-prints*, arXiv:1810.04805 (Oct. 2018). arXiv: 1810.04805 [cs.CL].
- [9] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *arXiv e-prints*, arXiv:2010.11929 (Oct. 2020). arXiv: 2010.11929 [cs.CV].
- [10] Jonathan Frankle and Michael Carbin. “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks”. In: *arXiv e-prints*, arXiv:1803.03635 (Mar. 2018). arXiv: 1803.03635 [cs.LG].

References III

- [11] Elias Frantar, Sidak Pal Singh, and Dan Alistarh. “Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning”. In: *arXiv e-prints*, arXiv:2208.11580 (Aug. 2022). arXiv: 2208.11580 [cs.LG].
- [12] Elias Frantar et al. “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”. In: *arXiv e-prints*, arXiv:2210.17323 (Oct. 2022). arXiv: 2210.17323 [cs.LG].
- [13] Dan Friedman, Alexander Wettig, and Danqi Chen. “Learning Transformer Programs”. In: *arXiv e-prints*, arXiv:2306.01128 (June 2023). arXiv: 2306.01128 [cs.LG].
- [14] Leo Gao et al. “The Pile: An 800GB Dataset of Diverse Text for Language Modeling”. In: *arXiv e-prints*, arXiv:2101.00027 (Dec. 2020). arXiv: 2101.00027 [cs.CL].
- [15] Catherine F. Higham, Desmond J. Higham, and Peter Grindrod. “Diffusion Models for Generative Artificial Intelligence: An Introduction for Applied Mathematicians”. In: *arXiv e-prints*, arXiv:2312.14977 (Dec. 2023). arXiv: 2312.14977 [cs.LG].
- [16] Jordan Hoffmann et al. “Training Compute-Optimal Large Language Models”. In: *arXiv e-prints*, arXiv:2203.15556 (Mar. 2022). arXiv: 2203.15556 [cs.CL].

References IV

- [17] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *arXiv e-prints*, arXiv:2106.09685 (June 2021). arXiv: 2106.09685 [cs.CL].
- [18] Jared Kaplan et al. “Scaling Laws for Neural Language Models”. In: *arXiv e-prints*, arXiv:2001.08361 (Jan. 2020). arXiv: 2001.08361 [cs.LG].
- [19] Dawid J. Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. “VeRA: Vector-based Random Matrix Adaptation”. In: *arXiv e-prints*, arXiv:2310.11454 (Oct. 2023). arXiv: 2310.11454 [cs.CL].
- [20] David Lindner et al. “Tracr: Compiled Transformers as a Laboratory for Interpretability”. In: *arXiv e-prints*, arXiv:2301.05062 (Jan. 2023). arXiv: 2301.05062 [cs.LG].
- [21] Ziming Liu and Max Tegmark. “A Neural Scaling Law from Lottery Ticket Ensembling”. In: *arXiv e-prints*, arXiv:2310.02258 (Oct. 2023). arXiv: 2310.02258 [cs.LG].
- [22] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. “Effective Approaches to Attention-based Neural Machine Translation”. In: *arXiv e-prints*, arXiv:1508.04025 (Aug. 2015). arXiv: 1508.04025 [cs.CL].

References V

- [23] Margaret Mitchell et al. “Model Cards for Model Reporting”. In: *arXiv e-prints*, arXiv:1810.03993 (Oct. 2018). arXiv: 1810.03993 [cs.LG].
- [24] C. Oliva and L.F. Lago-Fernández. “On the Interpretation of Recurrent Neural Networks as Finite State Machines”. In: *Artificial Neural Networks and Machine Learning – ICANN 2019: Theoretical Neural Computation*. Ed. by I. Tetko et al. Vol. 11727. Lecture Notes in Computer Science. Springer, Cham. 2019. DOI: 10.1007/978-3-030-30487-4_25.
- [25] OpenAI. “GPT-4 Technical Report”. In: *arXiv e-prints*, arXiv:2303.08774 (Mar. 2023). arXiv: 2303.08774 [cs.CL].
- [26] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *arXiv e-prints*, arXiv:2203.02155 (Mar. 2022). arXiv: 2203.02155 [cs.CL].
- [27] Alec Radford et al. *Improving Language Understanding by Generative Pre-Training*. OpenAI Blog, <https://openai.com/blog/language-unsupervised/>. June 2018. URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

References VI

- [28] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. OpenAI Blog, <https://openai.com/blog/better-language-models/>. Feb. 2019. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [29] Joseph Redmon and Ali Farhadi. “YOLOv3: An Incremental Improvement”. In: *arXiv e-prints*, arXiv:1804.02767 (Apr. 2018). arXiv: 1804.02767 [cs.CV].
- [30] Robin Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models”. In: *arXiv e-prints*, arXiv:2112.10752 (June 2022). International Conference on Computer Vision and Pattern Recognition (CVPR). New Orleans, LA. pp. 10684–10695. arXiv: 2112.10752 [cs.CV].
- [31] Hugo Touvron et al. “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *arXiv e-prints*, arXiv:2307.09288 (July 2023). arXiv: 2307.09288 [cs.CL].
- [32] Hugo Touvron et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv e-prints*, arXiv:2302.13971 (Feb. 2023). arXiv: 2302.13971 [cs.CL].

References VII

- [33] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- [34] Gail Weiss, Yoav Goldberg, and Eran Yahav. “Thinking Like Transformers”. In: *arXiv e-prints*, arXiv:2106.06981 (June 2021). arXiv: 2106.06981 [cs.LG].
- [35] Kelvin Xu et al. “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”. In: *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by Francis Bach and David Blei. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, July 2015, pp. 2048–2057. URL: <https://proceedings.mlr.press/v37/xuc15.html>.